

Министерство образования и науки Российской Федерации

Федеральное государственное бюджетное образовательное учреждение
высшего профессионального образования

ТОМСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ СИСТЕМ
УПРАВЛЕНИЯ И РАДИОЭЛЕКТРОНИКИ (ТУСУР)

Кафедра комплексной информационной безопасности
электронно-вычислительных систем (КИБЭВС)

Н.А. Новгородова

**СОЗДАНИЕ ПРОСТОГО ПРИЛОЖЕНИЯ
ДЛЯ РАБОТЫ С БАЗАМИ ДАННЫХ В MICROSOFT VISUAL STUDIO**

**Учебно-методические указания к выполнению лабораторных работ
по дисциплинам
«Безопасность систем баз данных»,
«Базы данных и экспертные системы»,
«Базы данных»**

Томск 2015

ЛАБОРАТОРНАЯ РАБОТА №2

СОЗДАНИЕ ПРОСТОГО ПРИЛОЖЕНИЯ ДЛЯ РАБОТЫ С БАЗАМИ ДАННЫХ В MICROSOFT VISUAL STUDIO

Цель работы:

- 1) Изучение основных компонент для работы с данными в базах данных в Microsoft Visual Studio;
- 2) Создание простого приложения в Microsoft Visual Studio для работы с базой данных для своей предметной области (на примере базы данных для кафе).

1 Введение

В предлагаемом лабораторном практикуме рассматриваются основные этапы проектирования автоматизированной информационной системы на учебном примере в среде Microsoft Visual Studio.

В качестве языка программирования в среде Microsoft Visual Studio для платформы Net Framework выбран C# (произносится Си Шарп).

Данный полнофункциональный объектно-ориентированный язык программирования специально для платформы .NET разработала компания Microsoft.

.NET Framework - программная технология от компании Microsoft, предназначенная для создания различных программ, в том числе WEB-приложений.

C# относится к семье языков с C-подобным синтаксисом, из них его синтаксис наиболее близок к C++ и Java.

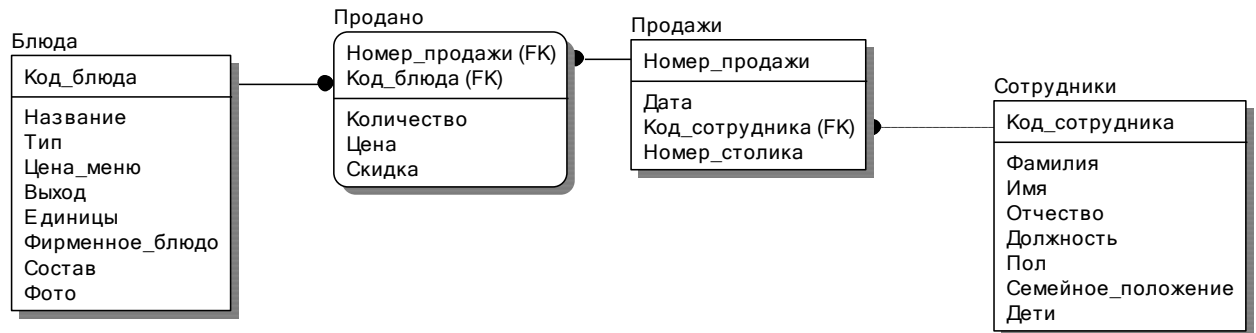
Язык C# находит широкое применение для разработки приложений для настольных и серверных Windows, Web-приложений, для мобильных платформ, таких как Windows Mobile и др.

В качестве учебного примера рассматривается пример реализации программы для кафе с использованием библиотек классов Microsoft.NET Framework, языка программирования C#, системы управления базами данных Microsoft SQL Server.

2 Информация о базе данных

Разрабатываемое приложение предназначено для работы с базой данных для кафе (см. лабораторную работу №1).

Структура БД для кафе в рассматриваемом примере:



В рассматриваемом приложении в качестве СУБД может использоваться Microsoft SQL Server.

Примечание:

В данном примере в БД для удобства объяснения и понимания предметной области используются названия таблиц и полей на русском языке.

3 Краткие теоретические сведения

В платформе .NET определено множество типов (организованных в соответствующие пространства имен) для взаимодействия с локальными и удаленными хранилищами данных. Общее название пространств имен с этими типами - ADO.NET.

ADO.NET - это технология доступа к базам данных, специально оптимизированная для нужд построения систем на платформе .NET.

Технология ADO.NET ориентирована на архитектуру многоуровневых приложений, которая в настоящее время стала фактическим стандартом для создания распределенных систем.

Основные отличительные особенности ADO.NET:

- ADO расширяет концепцию объектов-наборов записей в базе данных новым типом DataSet, который представляет локальную копию сразу множества взаимосвязанных таблиц. При помощи объекта DataSet пользователь может локально производить различные операции с содержимым базы данных, будучи физически рассоединен с СУБД, и после завершения этих операций передавать внесенные изменения в базу данных при помощи соответствующего "адаптера данных" (data adapter);

- в ADO.NET реализована поддержка представления данных в XML-совместимых форматах. В ADO.NET сформированные для локальной обработки наборы данных представлены в формате XML (в этом же формате они и передаются с сервера баз данных). Данные в форматах XML очень удобно передавать при помощи обычного HTTP, решает многие проблемы с установлением соединений через брандмауэры;

- ADO.NET - это библиотека управляемого кода и взаимодействие с ней производится как с обычной сборкой .NET. Типы ADO.NET используют возможности управления памятью CLR и могут использоваться во многих .NET - совместимых языках. При этом обращение к типам ADO.NET (и их членам) производится практически одинаково вне зависимости от того, какой язык используется.

Все типы ADO.NET предназначены для выполнения единого набора задач:

- установить соединение с хранилищем данных;
- создать и заполнить данными объект DataSet;
- отключиться от хранилища данных и вернуть изменения, внесенные в объект DataSet обратно в хранилище данных.

Объект DataSet - это тип данных, представляющий локальный набор данных и информацию об отношениях между ними.

На практике можно создать на клиенте объект DataSet, который будет представлять полную копию удаленной базы данных.

После создания объекта DataSet и его заполнения данными можно программными средствами производить запросы к нему и перемещаться по таблицам, выполнять все операции, как при работе с обычными базами

данных: добавлять в таблицы новые записи, удалять и изменять существующие, применять к ним фильтры и т.п. После того как клиент завершит внесение изменений, информация о них будет отправлена в хранилище данных для обработки.

В состав ADO.NET включены управляемые провайдеры для взаимодействия с различными СУБД.

В любом приложении ADO.NET необходимо использовать, по крайней мере, одно пространство имен - System.Data. В основном эти типы представляют собой объектные представления примитивов для работы с базами данных - таблицами, строками, столбцами, ограничениями и т. п. Наиболее часто используемые типы System.Data представлены ниже:

Тип	Назначение
DataColumnCollection, DataColumn	DataColumn представляет один столбец в объекте DataTable, DataColumnCollection - все столбцы
ConstraintCollection, Constraint	Constraint - объектно-ориентированная оболочка вокруг ограничения (например, внешнего ключа или уникальности), наложенного на один или несколько DataColumn, ConstraintCollection - все ограничения в объекте DataTable
DataRowCollection, Data Row	DataRow представляет единственную строку в DataTable, DataRowCollection - все строки в DataTable
DataRowView, Data View	DataRowView позволяет создавать настроенное представление единственной строки, DataView - созданное программным образом представление объекта DataTable, которое может быть использовано для сортировки, фильтрации, поиска, редактирования и перемещения
DataSet	Объект, создаваемый в оперативной памяти на клиентском компьютере. DataSet состоит из множества объектов DataTable и информации об отношениях между ними
ForeignKeyConstraint, UniqueConstraint	ForeignKeyConstraint представляет ограничение, налагаемое на набор столбцов в таблицах, связанных отношениями первичный - внешний ключ. UniqueConstraint - ограничение,

	при помощи которого гарантируется, что в столбце не будет повторяющихся записей
DataRelationCollection, DataRelation, DataTableCollection, DataTable	Тип DataRelationCollection представляет набор всех отношений (то есть объектов DataRelation) между таблицами в DataSet. Тип DataTableCollection представляет набор всех таблиц (объектов DataTable) в DataSet

В традиционных системах клиент-сервер при запуске приложения пользователем автоматически устанавливается связь с базой данных, которая поддерживается в "активном" состоянии до тех пор, пока приложение не будет закрыто. Такой метод работы с данными становится непрактичным, поскольку подобные приложения трудно масштабируются. Например, такая прикладная система может работать достаточно быстро и эффективно при наличии 8-10 пользователей, но она может стать полностью неработоспособной, если с ней начнут работать 100, 200 и более пользователей. Каждое открываемое соединение с базой данных "потребляет" достаточно много системных ресурсов сервера, они становятся занятыми поддержкой и обслуживанием открытых соединений, их практически не остается на процессы непосредственной обработки данных.

По этой причине, в ADO.NET используется модель работы пользователя в отрыве от источника данных. Приложения подключаются к базе данных только на небольшой промежуток времени. Соединение устанавливается только тогда, когда клиент удаленного компьютера запрашивает на сервере данные. После того, как сервер подготовил необходимый набор данных, сформировал и отправил их клиенту, связь приложения с сервером сразу же обрывается, и клиент просматривает полученную информацию уже не в связи с сервером.

В объектной модели ADO.NET можно выделить несколько уровней.

Уровень данных. Это по сути дела базовый уровень, на котором располагаются сами данные (например, таблицы базы данных СУБД). На данном уровне обеспечивается физическое хранение информации на магнитных носителях и манипуляция с данными на уровне исходных таблиц (выборка, сортировка, добавление, удаление, обновление и т. п.).

Уровень бизнес-логики. Это набор объектов, определяющих, с какой базой данных предстоит установить связь и какие действия необходимо будет выполнить с содержащейся в ней информацией. Для установления связи с базами данных используется объект DataConnection. Для хранения команд, выполняющих какие либо действия над данными, используется объект DataAdapter. И, наконец, если выполнялся процесс выборки информации из базы данных, для хранения результатов выборки используется объект DataSet. Объект DataSet представляет собой набор

данных "вырезанных" из таблиц основного хранилища, который может быть передан любой программе-клиенту, способной либо отобразить эту информацию конечному пользователю, либо выполнить какие-либо манипуляции с полученными данными.

Уровень приложения. Это набор объектов, позволяющих хранить и отображать данные на компьютере конечного пользователя. Для хранения информации используется уже знакомый нам объект DataSet, а для отображения данных имеется довольно большой набор элементов управления (DataGridView, TextBox, ComboBox и т. д.).

В Visual Studio .Net можно вести разработку приложений различных типов. В первую очередь это традиционные Windows-приложения (на основе Windows-форм), которые реализованы в виде exe-файлов, запускаемых на компьютере пользователя. Ну и конечно, Web-приложения (на основе Web-форм), которые работают в оболочке браузера. Для хранения данных на уровне обоих типов приложений используется объект DataSet.

Обмен данными между приложениями и уровнем бизнес-логики происходит с использованием формата XML, а средой передачи данных служат либо локальная сеть (Интранет), либо глобальная сеть (Интернет).

В ADO.NET для манипуляции с данными могут использоваться команды, реализованные в виде SQL-запросов или хранимых процедур (DataCommand).

Если требуется выполнить более чем одну операцию с данными, например, получить некоторый набор данных, а затем скорректировать его, - то необходимо выполнить последовательность команд. Каждая команда выполняется отдельно, последовательно одна за другой. Между выполняемыми командами соединение с базой отсутствует.

Очень часто, в приложении нужно обработать не одну запись, а их набор: список сотрудников, перечень продаж и т.п. Как правило, в приложениях требуется одновременная работа с более чем одной таблицей: сотрудники и все их продажи; продажа и ее элементы, т.е. с набором связанных данных. Причем для удобства пользователя данные требуется группировать и сортировать то по одному, то по другому признаку. При этом нерационально каждый раз возвращаться к исходной базе данных и заново считывать данные. Более практично работать с некой временной "вырезкой" информации, хранящейся в оперативной памяти компьютера.

Эту роль выполняет набор данных - DataSet, который представляет собой своеобразный кэш записей, извлеченных из базового источника. DataSet может состоять из одной или более таблиц, он имеет дело с копиями таблиц из базы данных источника. Кроме того, в данном объекте могут содержаться связи между таблицами и некоторые ограничения на выбираемые данные:

Данные в DataSet - это некий уменьшенный вариант основной базы данных. Тем не менее, вы можете работать с такой "вырезкой" точно так же, как и с реальной базой.

Вы можете выполнить операции коррекции непосредственно в DataSet, а потом все внесенные изменения будут переданы в основную базу данных.

Важно отметить то, что DataSet - пассивный контейнер для данных, который обеспечивает только их хранение. Что же нужно поместить в этот контейнер, определяется в другом объекте - адаптере данных DataAdapter. В адаптере данных содержатся одна или более команд, которые определяют, какую информацию нужно поместить в таблицы объекта DataSet, по каким правилам нужно синхронизировать информацию в конкретной таблице DataSet и соответствующей таблицей основной базы данных и т. п.

Адаптер данных обычно содержит четыре команды SELECT (выбрать), INSERT (добавить), UPDATE (изменить), DELETE (удалить).

Набор данных DataSet - "независимая" копия фрагмента базы данных, расположенная на компьютере пользователя. Причем в этой копии могут быть не отражены те изменения, которые могли внести в основную базу данных другие пользователи. Если требуется увидеть самые последние изменения, сделанные другими пользователями, то необходимо обновить DataSet.

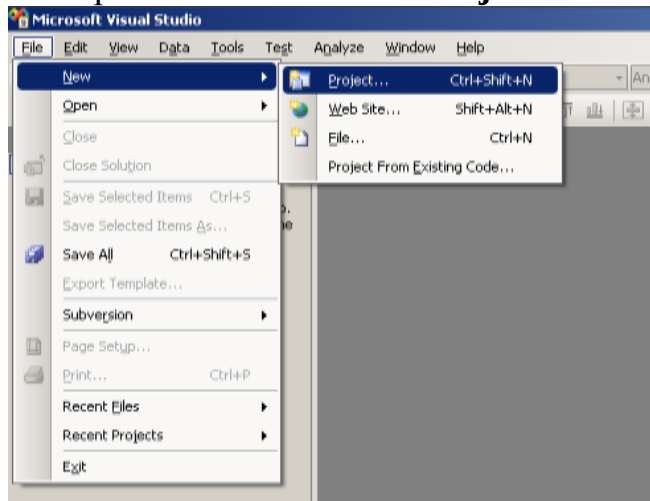
4 Практические задания

4.1 Создание приложения для работы с данными кафе

Создайте на диске отдельную папку для файлов дальнейшей программы.
Например, C:\LabsBD\

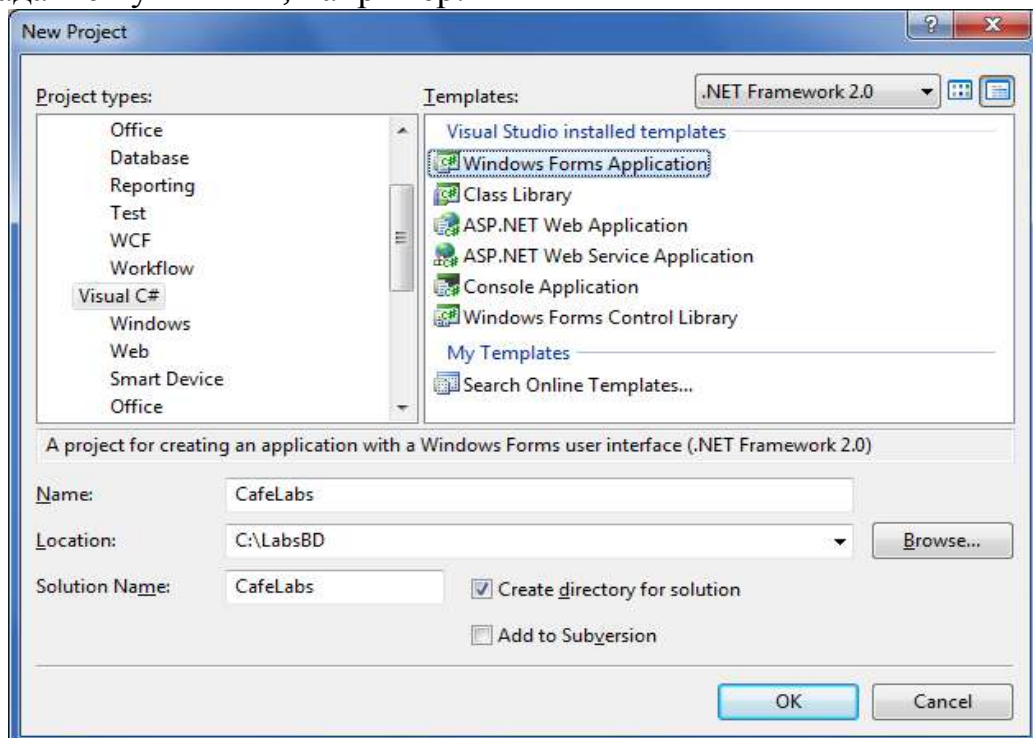
Запустите **Microsoft Visual Studio**.

Выберите меню **File\New\Project**.



Выберите проект на Visual C#.

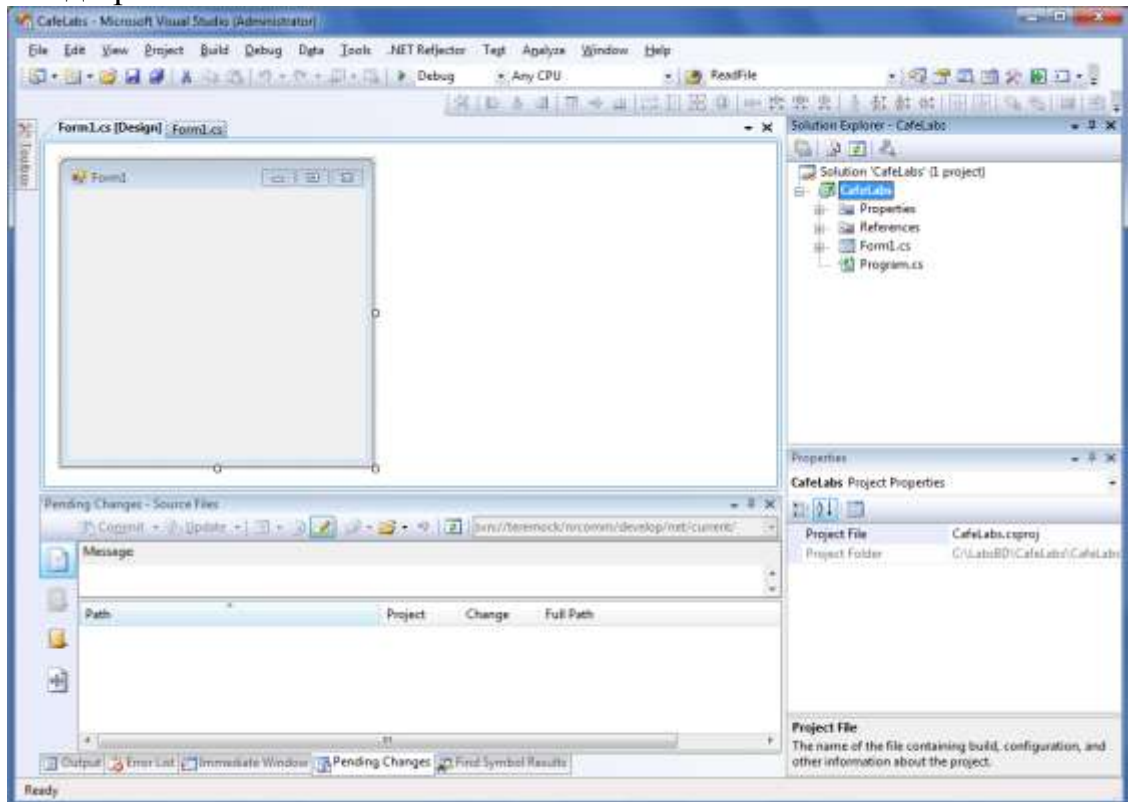
Задайте путь и имя, например:



Нажмите **OK**.

Установленный флаг **Create directory for solution** приведет к тому, что для файлов проекта будет создана отдельная папка, с соответствующим именем.

Вид проекта в Visual Studio:



При помощи дизайнера графических форм можно добавлять в приложение различные элементы управления и он будет автоматически генерировать код для этих элементов (по умолчанию файл с главной формой приложения называется Form1.cs).

В Visual Studio классы Windows-форм формируются в двух файлах: Form1.cs и Form1.Designer.cs. В файле Form1.Designer.cs присутствует код, сгенерированный дизайнером, а файле Form1.cs - присутствует код инициализации класса и пользовательские члены класса (поля, свойства, методы, события и др.)

Для разметки окон приложения в соответствии с требованиями пользователя необходимо изменить свойства класса Forms1. Это можно сделать с помощью дизайнера окон (**Form Designer**), путем изменения свойств в окне Свойства (**Properties**) или в коде программы.

Наш проект будет состоять из нескольких форм.

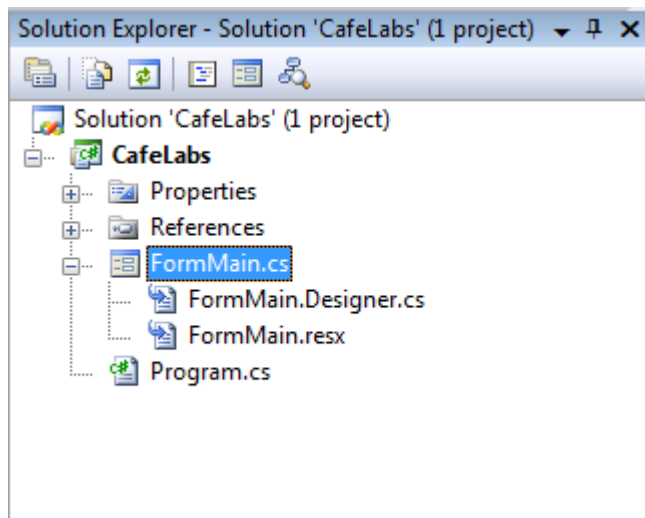
4.2. Подготовка главной формы

Данная форма отображается после запуска программы.

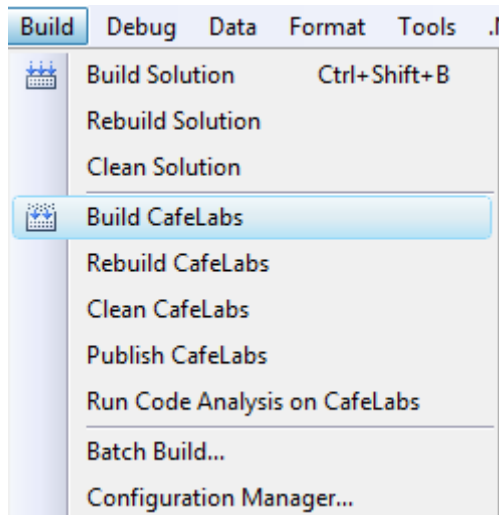
На вкладке **Properties** (Свойства) для **Form1** измените значение в поле **Text** (Заголовок) на **Программа для кафе**. При этом на форме изменится заголовок окна.

На вкладке **Properties** (Свойства) измените для **Form1** значение в свойстве **Name** (Имя) на **FormMain**.

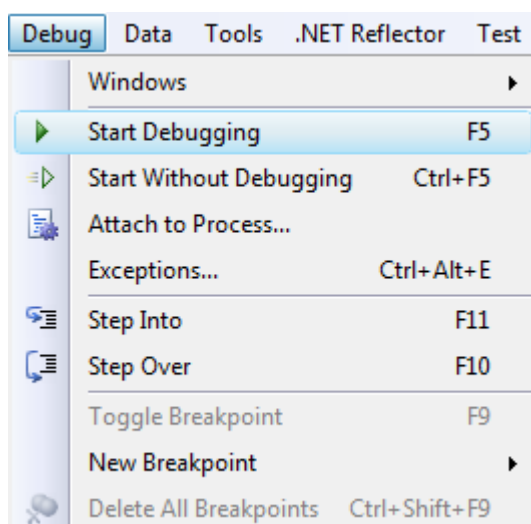
В окне проекта измените имя файла для главной формы на **FormMain.cs**:



Откомпилируйте приложение, выбрав из главного меню раздел **Build** действие **Build CafeLabs**:

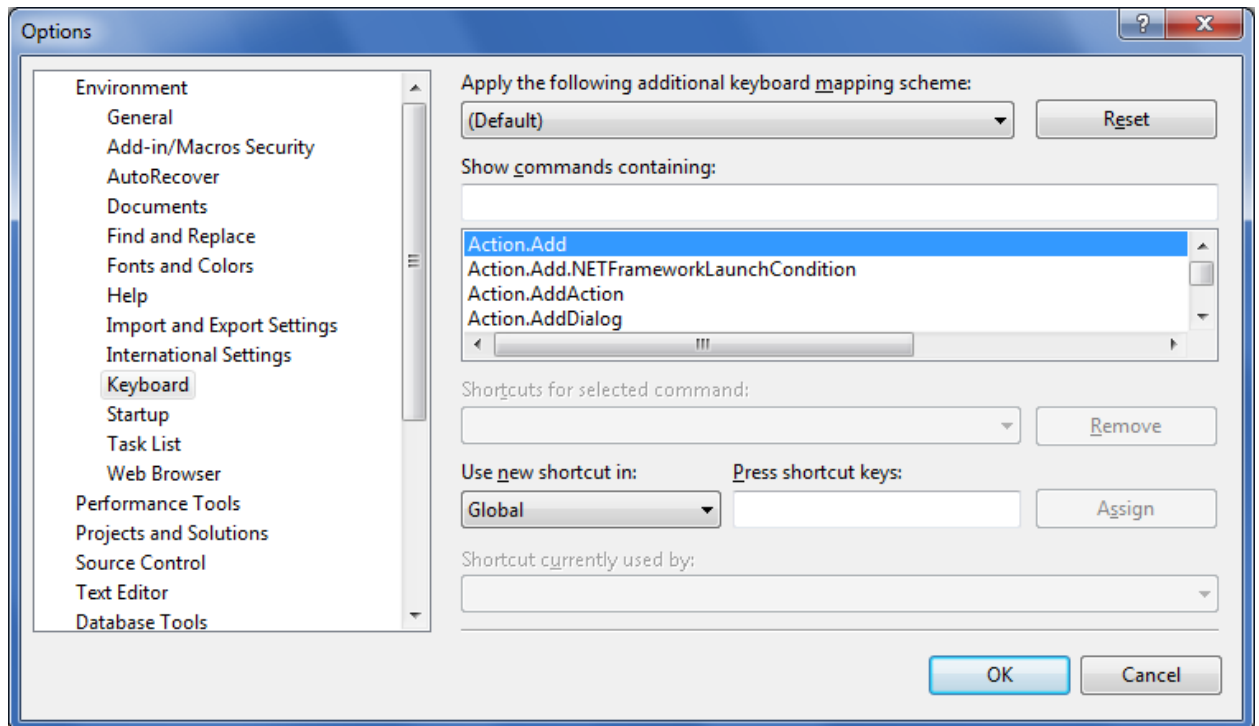


Нажмите **F5**. Посмотрите, как работает созданное приложение.



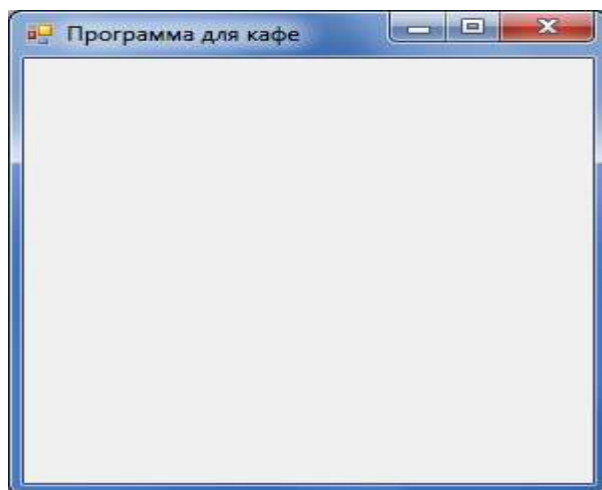
Примечание:

Описание горячих клавиш дается согласно настройкам по умолчанию в Visual Studio:



Если Вы их в настройках меняете, то они могут быть другими.

Вид формы запущенного приложения имеет вид:



Закройте запущенную программу. Перейдите Visual Studio.

4.2.1 Создание главного меню

Для создания главного меню используется компонент **MenuStrip** с вкладки **ToolBox**.

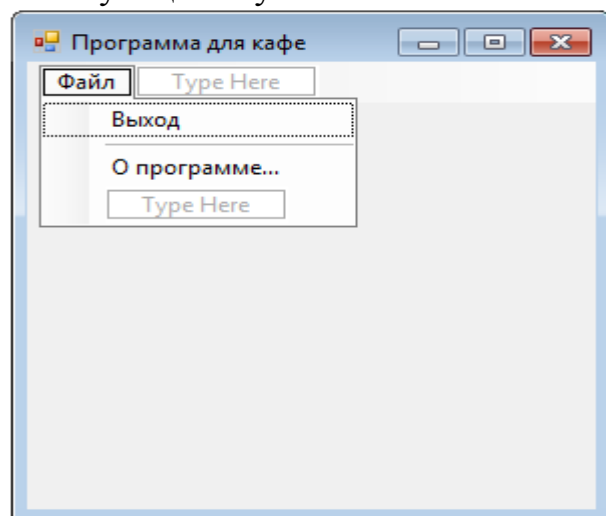
Это невидимый компонент при запуске приложения. При запуске приложения пользователи видят не сам компонент, а меню, сгенерированное им.

Создание главного меню на главной форме.

Добавим в главную форму элемент для меню. Для этого откройте вкладку **ToolBox** и сначала щелкните мышью на элементе **MenuStrip** вкладки, а затем щелкнем мышью на форме.

Свойство **Name** для него укажите в **menuStripMain**

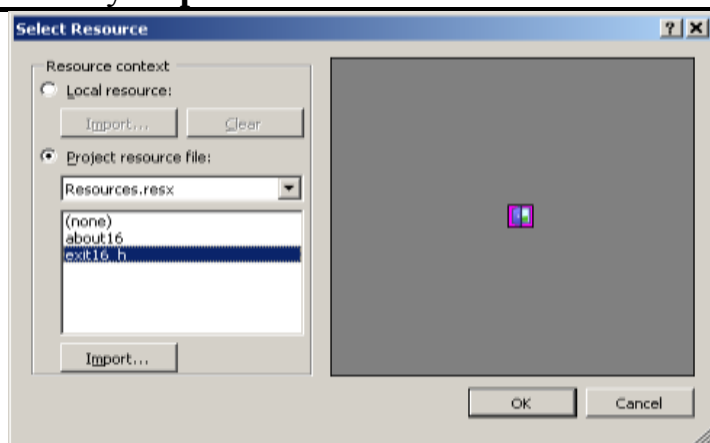
В дизайнера меню укажите пункты через свойство **Text** соответствующего пункта:



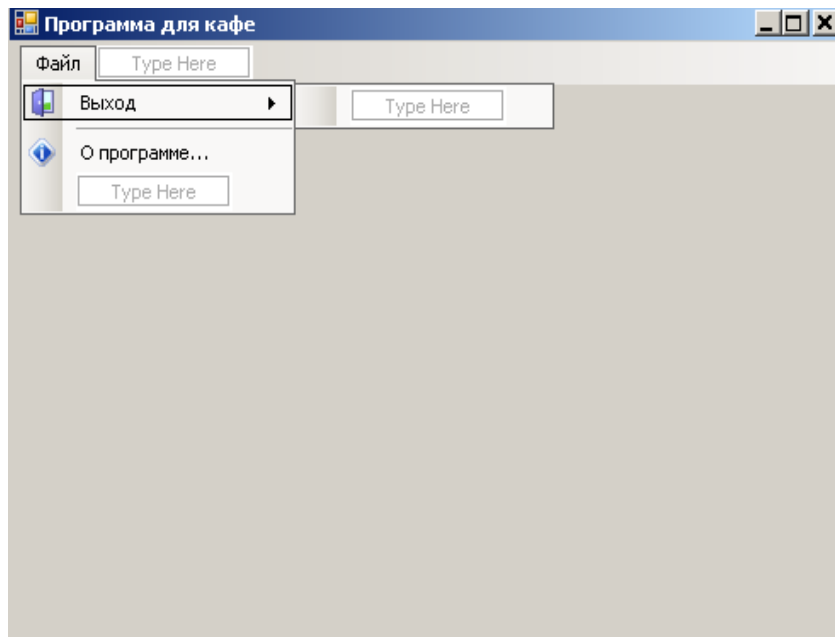
menuStripMain

Для данных пунктов задайте пиктограммы через свойство **Image**.

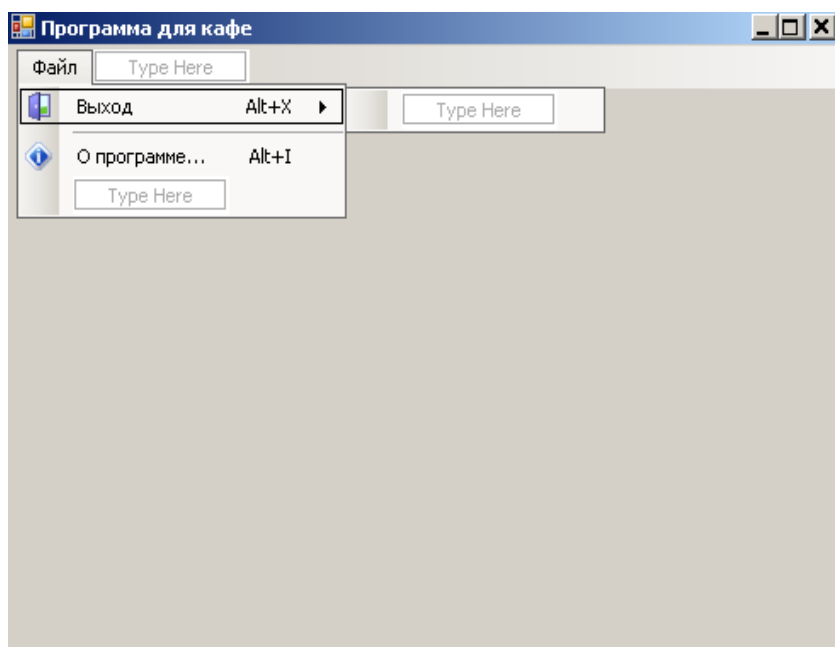
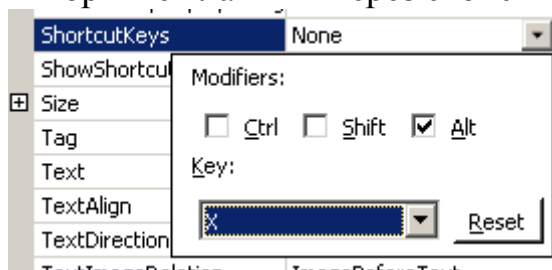
Сначала добавьте понравившиеся пиктограммы через **Project resource file** и кнопку **Import**:



Затем укажите выбранную пиктограмму соответствующему пункту меню:



и горячие клавиши через свойство **ShortcutKeys**, например:



Запрограммируем данные пункты меню.

Нажмите два раза мышкой на пункте меню **Выход** и в обработке события **Click** (клик мышью) пропишите, вызов метода **Close()**.

```
private void выходToolStripMenuItem_Click(object sender, EventArgs e)
```

```
{
    Close();
}
```

Примечание:

Рассмотрим код данного листинга.

Первая строка является объявлением метода, сгенерированным Visual Studio на языке C#.

Первое слово **private** определяет видимость метода, как внутренний, т.е. видимый только для членов класса `Main_Form`.

Второе слово **void** определяет тип данных возвращаемого значения (результата). Ключевое слово **void** перед именем функции, или в качестве аргумента функции (в скобках в конце строки), означает отсутствие соответствующего элемента.

Третье слово **выходToolStripMenuItem_Click** обозначает имя метода. За именем следует список передаваемых аргументов, заключенный в круглые скобки. Круглые скобки нужно использовать всегда, даже когда у функции нет параметров.

Тело функции заключается в фигурные скобки **{}**

Последняя строка – это вызов метода `Close()`, позволяющего закрыть форму нашего приложения (в данном случае главную форму):

Close();

C# чувствителен к регистру. В именах функций и переменных заглавные (прописные) буквы должны использоваться точно так же, как в их объявлениях.

Две косые черты, или слэши (**//**) - все, что следует до конца строки после двух идущих подряд косых черт (без пробелов, табуляций и т.п. между ними) рассматривается компилятором как комментарий и игнорируется.

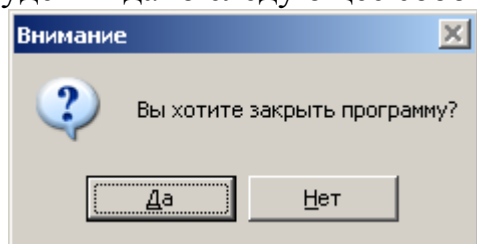
При добавлении в код нескольких строк комментариев используются символы в начале комментария **/***, а в конце комментария ***/**.

Данный метод **Close()** может вызываться не только в данном меню **Выход (выходToolStripMenuItem)**, но и когда пользователь нажмет **✕** на форме.

Запрограммируем для данного метода в событие (вкладка **Events**) **FormClosing** у **FormMain** вызов диалога, подтверждающего закрытие программы:

```
private void FormMain_FormClosing(object sender, FormClosingEventArgs e)
{
    e.Cancel = MessageBox.Show("Вы хотите закрыть программу?",
    "Внимание", MessageBoxButtons.YesNo, MessageBoxIcon.Question) !=
    DialogResult.Yes;
}
```

В результате, выбирая пункт меню **Выход** или закрывая программу по **х**, будет выдано следующее сообщение:



При нажатии Да, программа будет закрыта.

Примечание:

Для метода **MessageBox.Show()**

Перечисление MessageBoxButtons	
Константа	Кнопки, отображаемые в окне MessageBox
OK	OK
OKCancel	OK, Cancel
YesNo	Yes, No
YesNoCancel	Yes, No, Cancel
RetryCancel	Retry, Cancel
AbortRetryIgnore	Abort, Retry, Ignore
Перечисление MessageBoxIcon	
Константа	Значок
Asterisk, Information	
Error, Stop	
Exclamation, Warning	
Question	
None	Значок не отображается

Если в окне диалогового окна **MessageBox** имеется несколько кнопок, то для того, что бы определить, какую кнопку щелкнул пользователь, программа должна проанализировать значение, возвращенное методом **MessageBox.Show()**. Метод **MessageBox.Show()** может вернуть одно из нескольких значений перечисления **DialogResult**:

Перечисление DialogResult	
Константа	Кнопка, при щелчке которой возвращается эта константа
Abort	Abort
Cancel	Cancel
Ignore	Ignore
No	No

None	Модальное диалоговое окно продолжает работать
OK	OK
Retry	Retry
Yes	Yes

Продолжаем программировать пункты меню.

Пункт **О программе...**:

```
private void oПрограммеToolStripMenuItem_Click(object sender, EventArgs e)
{
    MessageBox.Show("(С) ТУСУР, КИБЭВС, ФИО, группа, 2015", "О программе",
    MessageBoxButtons.OK, MessageBoxIcon.Information);
}
```

Примечание:

Укажите свои ФИО и группу.

Примечание:

Согласно «Гражданскому кодексу Российской Федерации (часть четвертая)» к охраняемым результатам интеллектуальной деятельности и средствам индивидуализации (статья 1225), которым предоставляется правовая охрана (интеллектуальной собственностью), относят программы для электронных вычислительных машин (**программы для ЭВМ**), **базы данных**.

К объектам авторских прав относятся программы для ЭВМ, которые охраняются как литературные произведения (статья 1259).

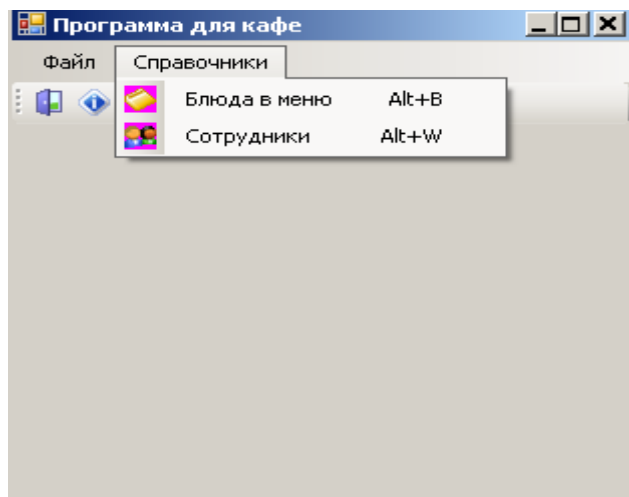
Согласно статье 1271, правообладатель для оповещения о принадлежащем ему исключительном праве на произведение вправе использовать **знак охраны авторского права, который состоит из следующих элементов:**

- латинской буквы "С" в окружности или скобках;
- имени или наименования правообладателя;
- года первого опубликования произведения.

Таким образом, знак охраны авторского права в данном случае может иметь вид:

(С)ТУСУР, КИБЭВС, ФИО, группа, 2015.

По аналогии определите пункты меню для справочников блюд, сотрудников:



4.2.2 Создание контекстного (локального, всплывающего) меню

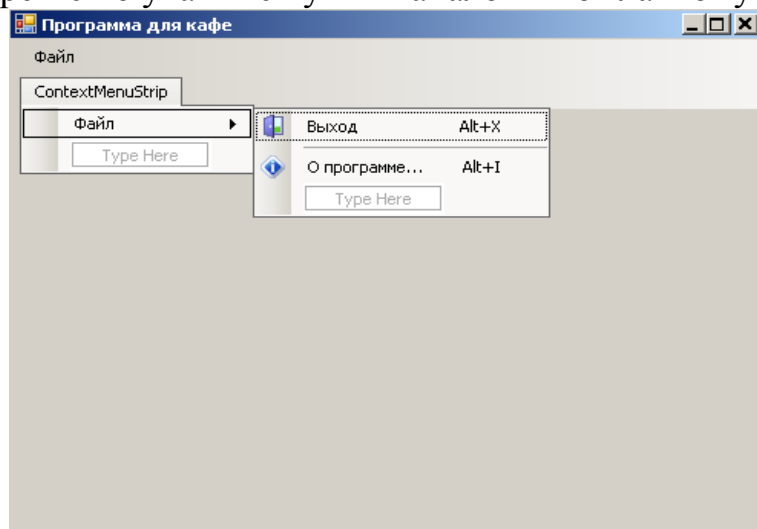
Контекстное (локальное, всплывающее) меню обычно появляется на экране при нажатии правой кнопки мышки на визуальных компонентах.

Для контекстного меню используется элемент **ContextMenuStrip** с закладки **ToolBox**.

Добавим в главную форму элемент для контекстного меню. Для этого откройте вкладку **ToolBox** и сначала щелкните мышью на элементе **ContextMenuStrip** вкладки, а затем щелкнем мышью на форме.

Свойство **Name** для него укажите в **contextMenuStripMain**.

В дизайнера меню укажите пункты аналогично главному меню:



Для обработчиков данных пунктов меню через событие **Click** укажите соответственно обработчики:

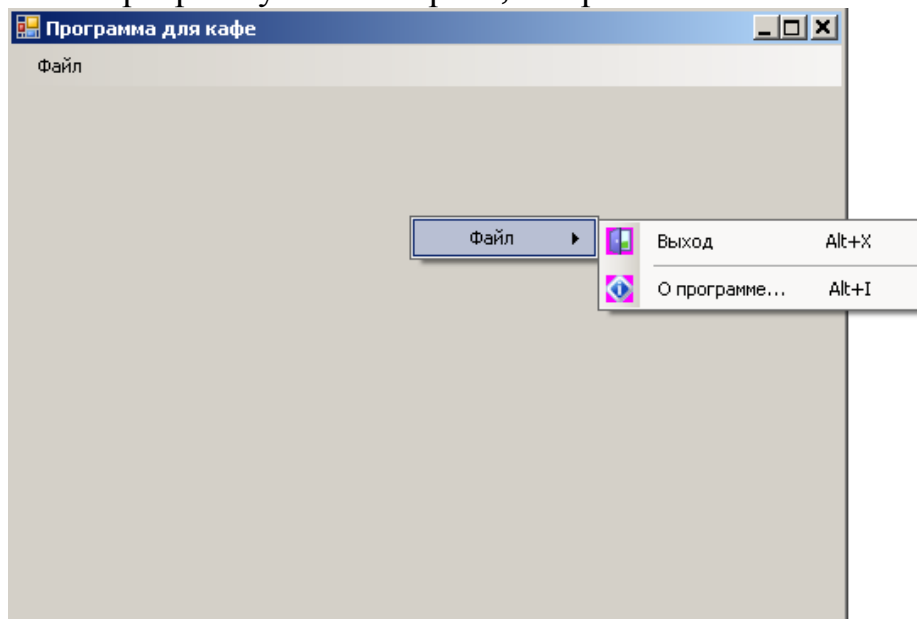
выходToolStripMenuItem_Click

оПрограммеToolStripMenuItem_Click

чтобы дважды данные действия не программировать.

Чтобы контекстное меню появлялось при нажатии правой кнопки мыши на форме, укажите в свойстве **contextMenuStrip** формы **FormMain** созданное **contextMenuStripMain**.

Запустите программу и посмотрите, как работает контекстное меню.



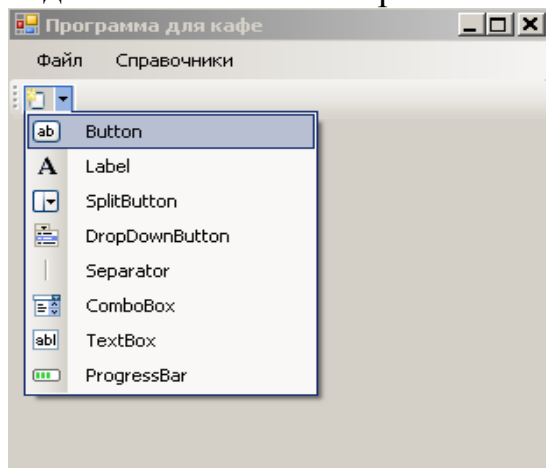
Аналогично сделайте для справочников блюд, сотрудников.

4.2.3 Создание панели инструментов

Для панели инструментов используется элемент **ToolStrip** с закладки **ToolStrip**.

Разместите элемент **ToolStrip** на форму. Укажите для него свойство **Name** в **toolStripMain**.

В дизайне панели выберете **Button**:

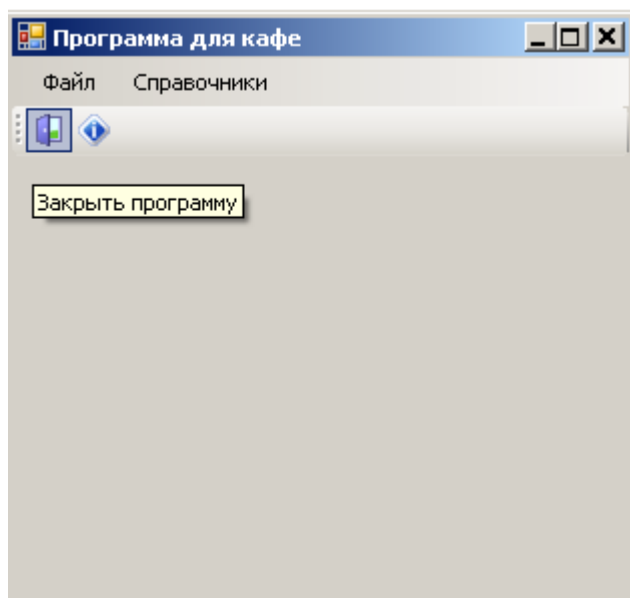


Для нее укажите аналогичную пиктограмму для выхода и всплывающую подсказку (через свойство **ToolTipText**).

В событии **Click** укажите для данной кнопки соответственно:

выходToolStripMenuItem_Click

Аналогично создайте кнопку для **О программе**.



Аналогично сделайте для справочников блюд и сотрудников.

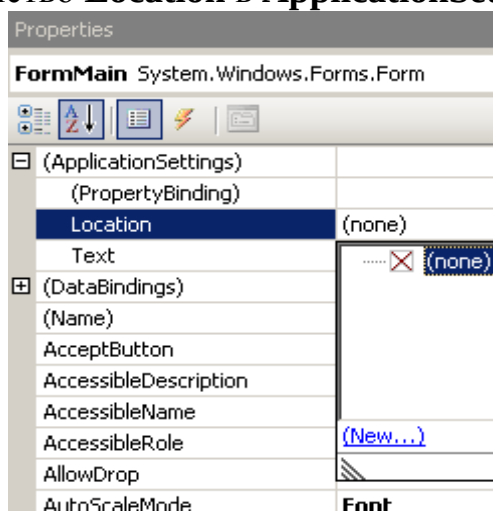
Запустите программу. Посмотрите, как отрабатывают созданные обработчики.

Закройте программу. Вернитесь в Visual Studio.

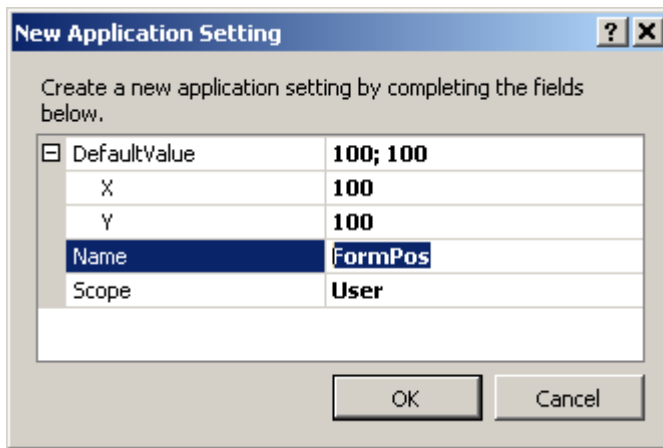
Примечание:

Существует возможность автоматического сохранения состояния большинства свойств элементов управления в конфигурационном файле приложения.

Например, для того, чтобы форма запущенного приложения сохранила свое положение на экране в конфигурационном файле и восстановила при следующем запуске свое положение на экране, необходимо изменить свойство **Location** в **ApplicationSettings** формы **FormMain**:

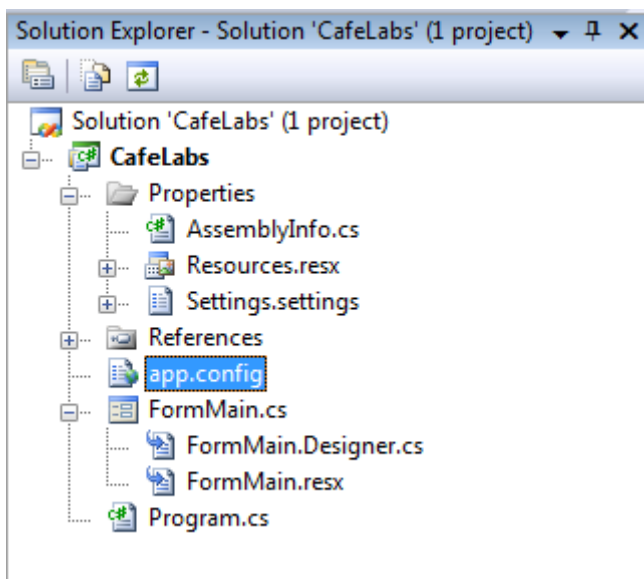


Нажмите **New** и укажите в появившемся диалоговом окне название данной настройки (**FormPos**), которая будет использована в конфигурационном файле, и значения позиции формы по умолчанию:



Нажмите ОК.

Теперь в составе проекта появился конфигурационный файл **app.config**



В проект при этом нужно добавить следующую строку кода в событии закрытия главной формы приложения:

```
private void FormMain_FormClosed(object sender, FormClosedEventArgs e)
{
    Settings.Default.Save();
}
```

При этом необходимо добавить пространство имен:

```
using CafeLabs.Properties;
```

В процессе работы приложения изменяемые настройки программы для которых указано **Scope** в **User** будут сохранены в конфигурационный файл, расположенный в домашней папке пользователя. Свойства и параметры имеющие признак **Scope** равный **Application** хранятся в файле **CafeLabs.exe.config** расположенном рядом с исполняемым файлом. В данном примере в папке:

C:\LabsBD\CafeLabs\CafeLabs\bin\Debug\CafeLabs.exe.config

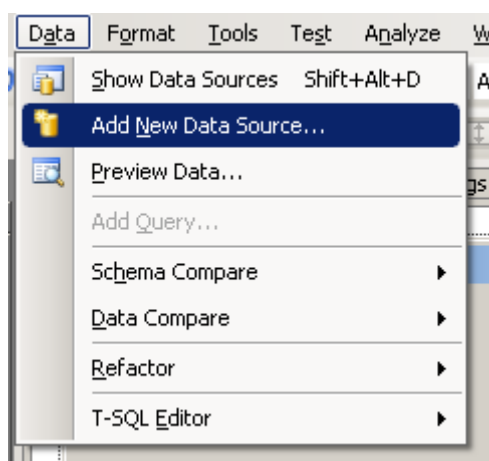
Запустите приложение. Проверьте настроенную возможность главной формы запоминать свое положение на экране.

4.3. Подключение к БД

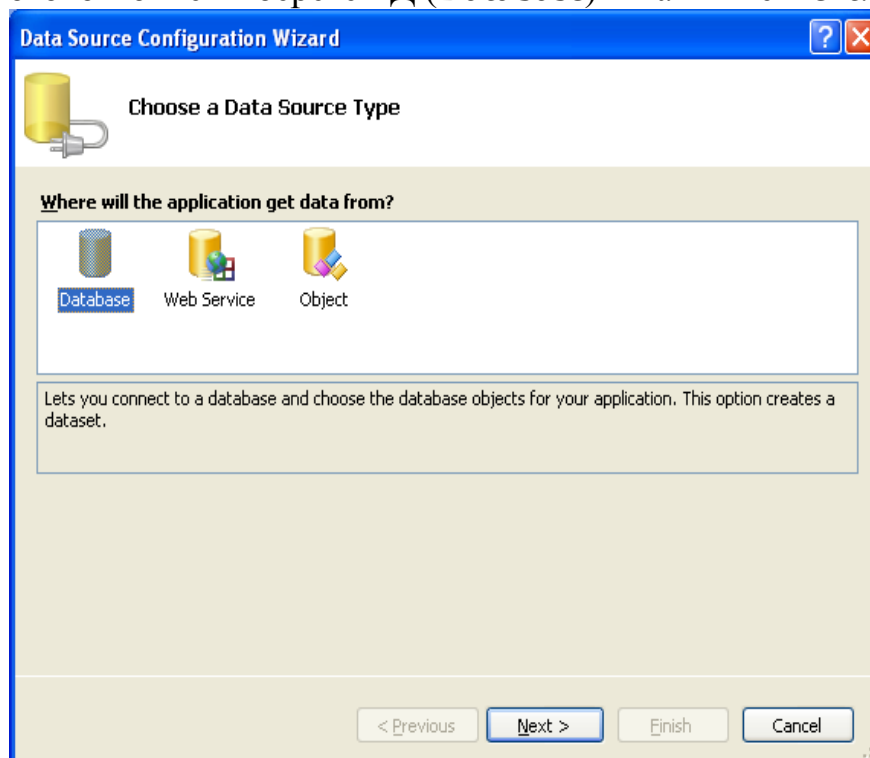
Программа для кафе должна позволять работать с данными базы данных.

Для этого необходимо подключиться к базе данных в приложении.

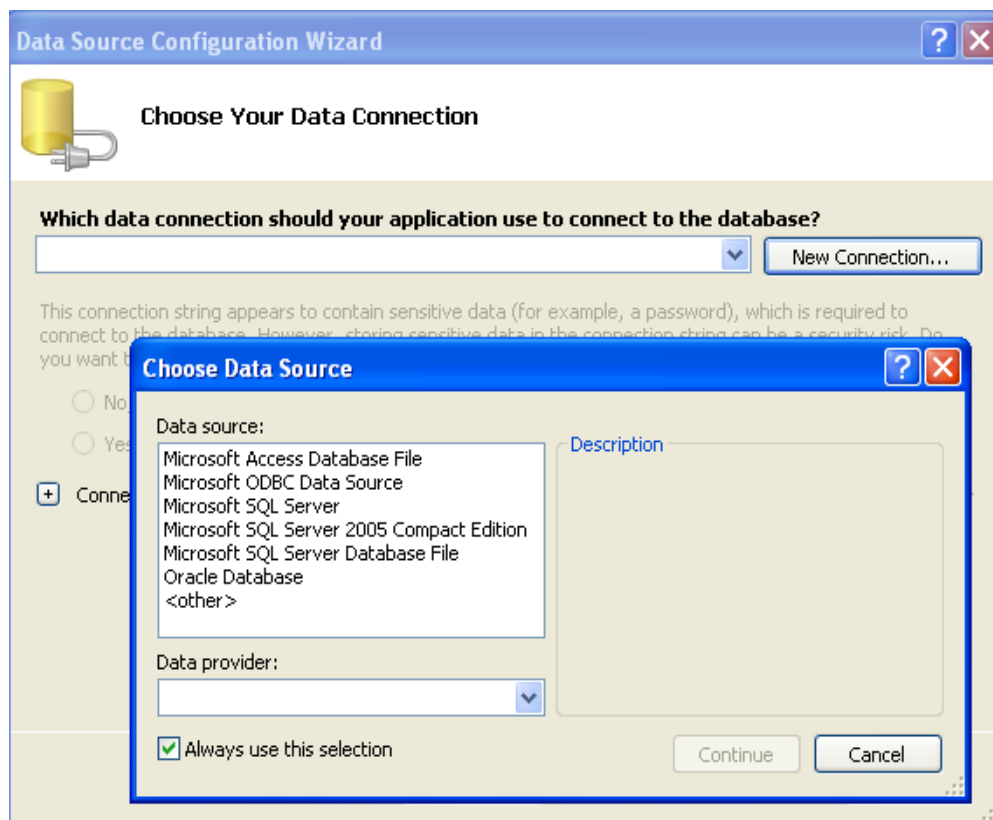
Выберете меню **Data\Add New Data Source**:



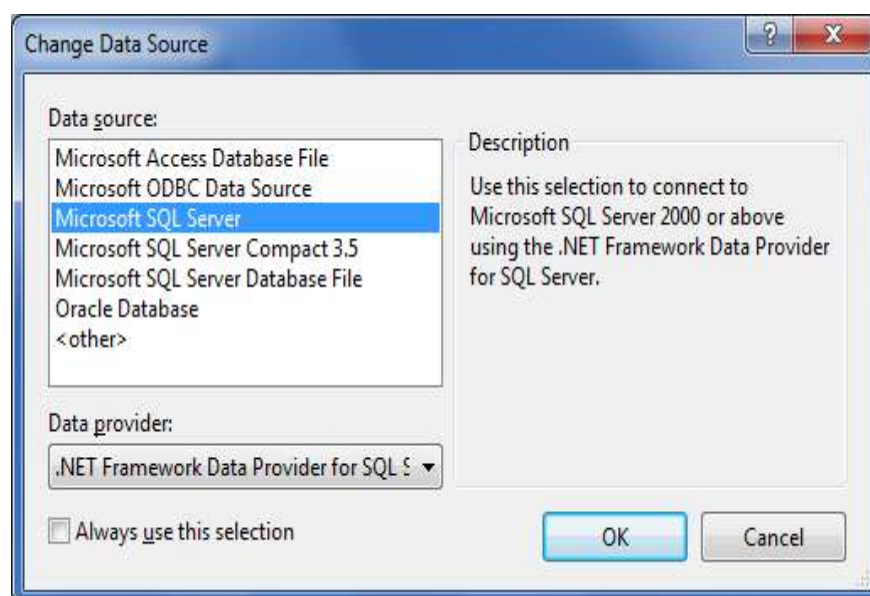
В диалоговом окне выберете БД (**Database**) и нажмите **Next**:



Выберете Новое соединение (**New connection**), после этого откроется диалоговое окно, в котором необходимо выбрать источник данных.



Для БД под управлением СУБД Microsoft SQL Server выберете **Microsoft SQL Server**:

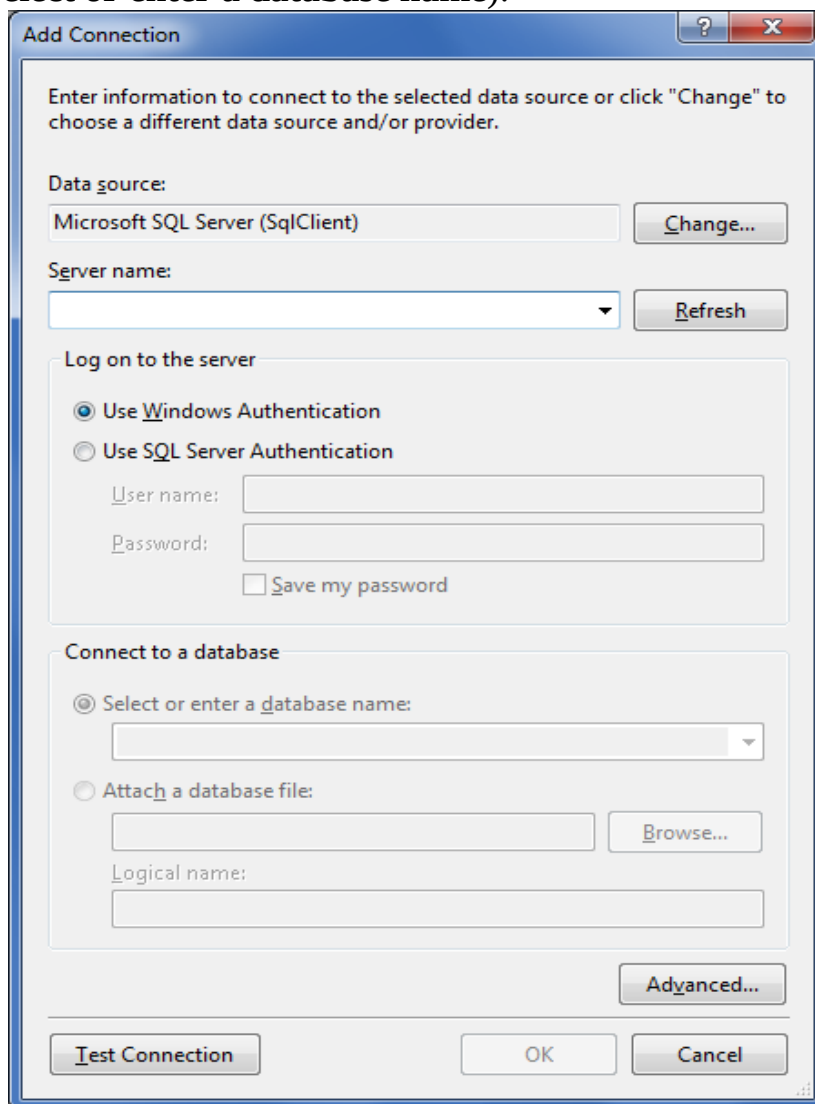


Если у Вас другая СУБД, то выберете необходимую СУБД.

Нажмите кнопку **ОК** для продолжения формирования источника данных.

Для СУБД Microsoft SQL Server:

Далее укажите имя сервера (**Server name**), режим необходимой аутентификации (**Log on to the server**) (см. лабораторную работу №1), имя базы данных (**Select or enter a database name**):



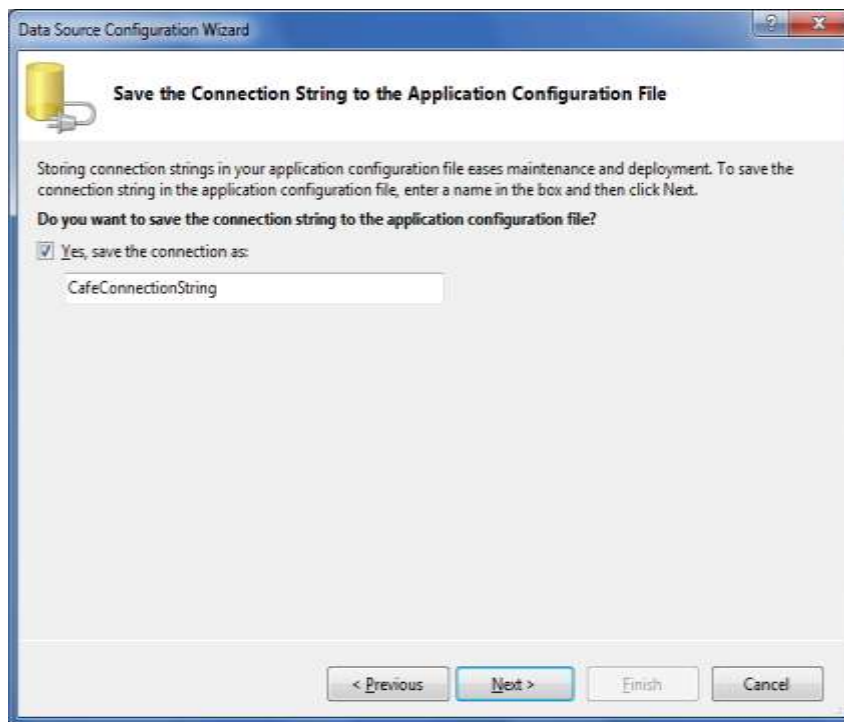
Можете нажать **Test Connection** для проверки соединения с БД. Если соединение прошло успешно, то выдается сообщение:



Затем нажмите кнопку **OK** и продолжите формировать источник данных.

Нажмите кнопку **Next**.

Укажите необходимость создания конфигурационного файла, так как путь к БД может меняться и чтобы не перекомпилировать программу при смене пути к БД, достаточно поменять путь в конфигурационном файле.

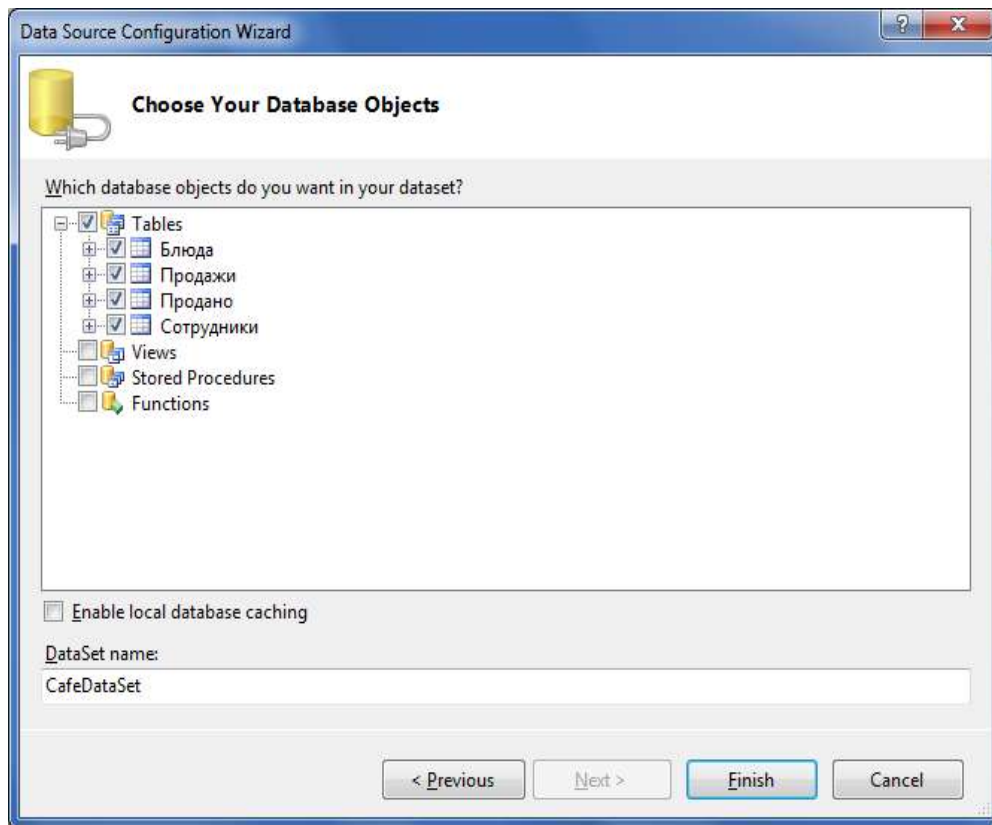


Нажмите **Next**.

Далее укажите объекты, которые будут помещены в создаваемый набор данных (**DataSet**).

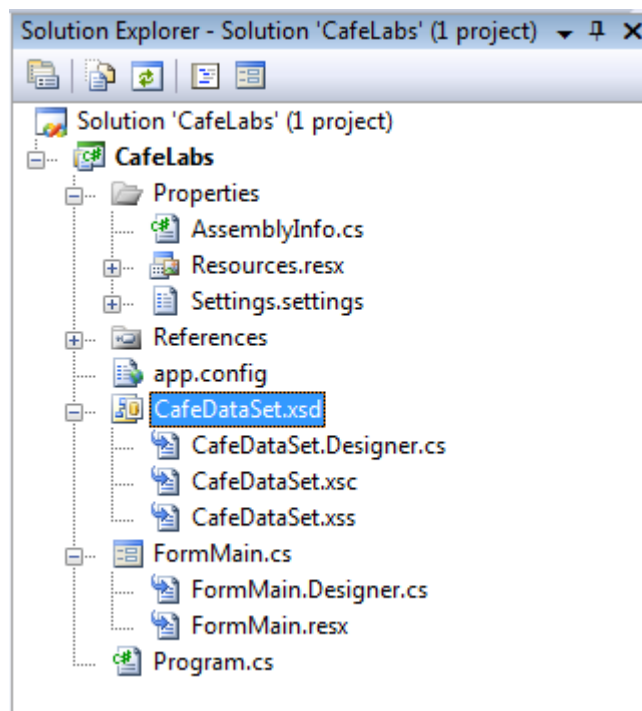
Наша программа будет работать со всеми таблицами БД, которые и добавим в набор данных.

Если нужны еще объекты БД, кроме таблиц, то необходимо указать и их.



Нажмите кнопку **Finish**.

В результате будет создан класс **CafeDataSet**, который добавляется в решение проекта:



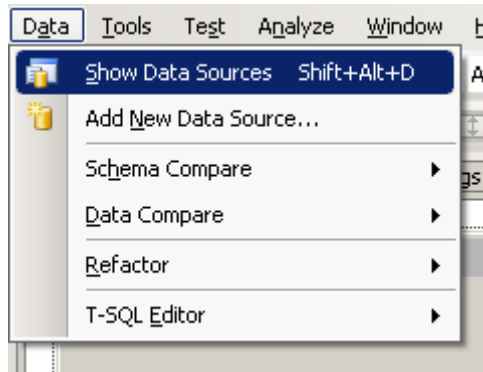
Примечание:

Конфигурационный файл находится в папке, где .exe файл и имеет имя (в данном примере):

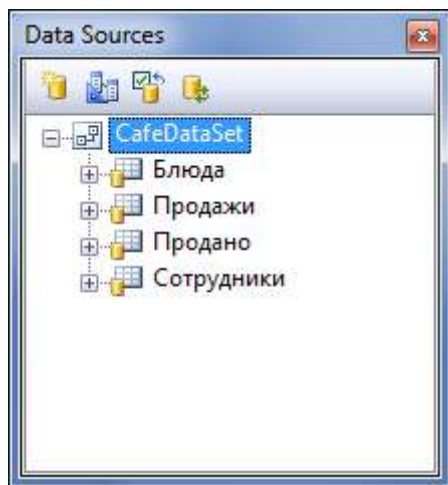
C:\LabsBD\CafeLabs\CafeLabs\bin\Debug\CafeLabs.exe.config

Таким образом, нами сформирован набор данных **DataSet**, являющийся источником данных (**DataSource**) в приложении.

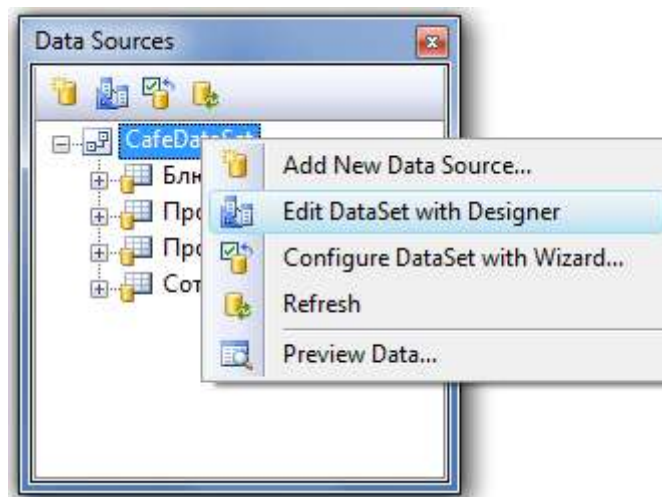
Выберете меню **Data**, а затем **Show Data Sources**:



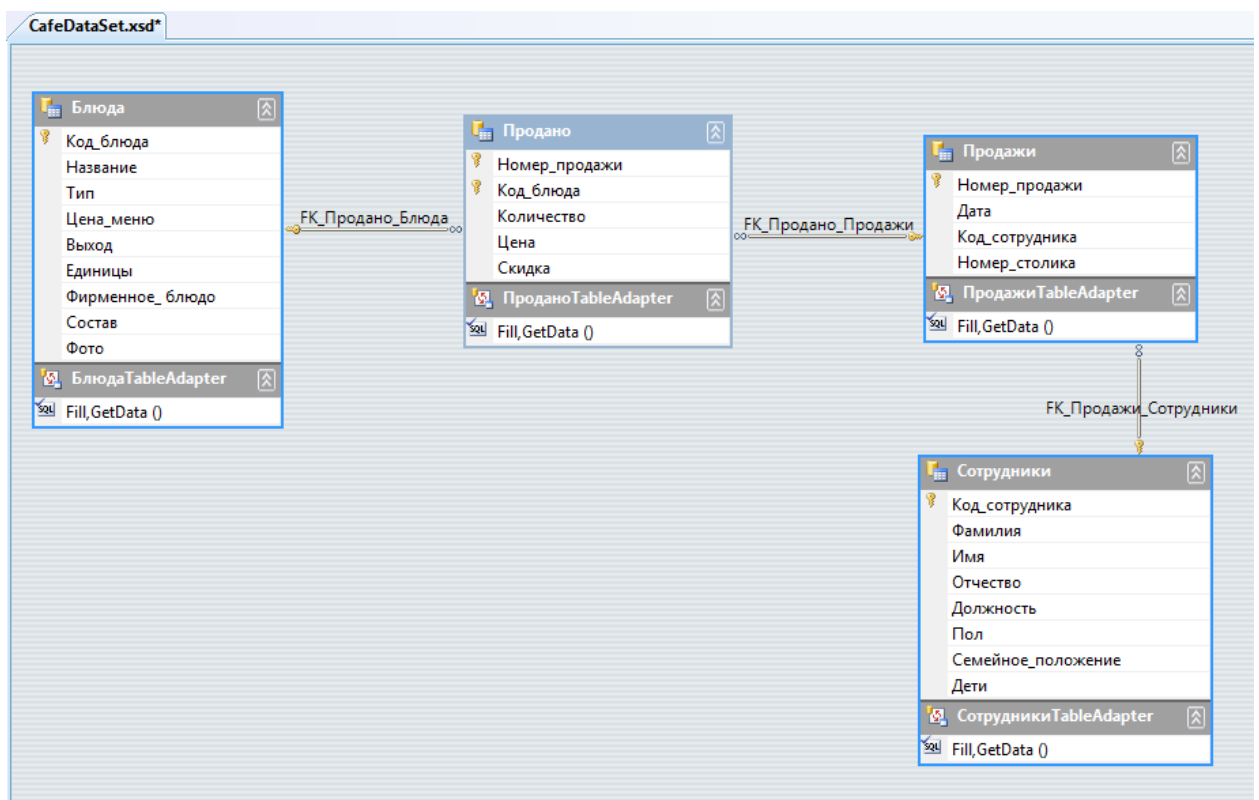
Для просмотра сформированного источника данных:



Если нажать правую кнопку мыши на сформированном наборе данных **CafeDataSet** и выбрать **Edit Dataset with Designer**:



То Вы сможете просмотреть дизайнер сформированного набора данных (или нажмите 2 раза мышкой на наборе данных **CafeDataSet.xsd**):



Как видно сформированы классы таблиц, адаптеры и методы **Fill** и **GetData**:

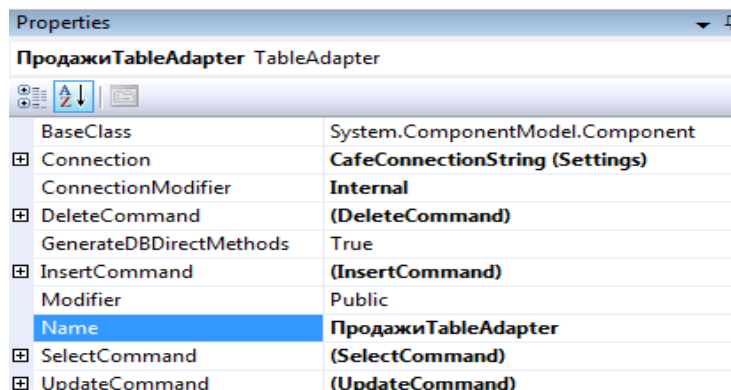
Блюда, Сотрудники, Продажи, Продано типа **DataTable**;

БлюдаTableAdapter, СотрудникиTableAdapter, ПродажиTableAdapter, ПроданоTableAdapter типа **DataAdapter**.

Как было сказано ранее, объекты типа **DataAdapter** позволяют проводить основные операции в наборе данных соответствующих объектов БД.

TableAdapter позволяет работать с данными в наборе данных соответствующих таблиц БД через соответствующие свойства.

Например, для **ПродажиTableAdapter**:



Properties	
ПродажиTableAdapter TableAdapter	
BaseClass	System.ComponentModel.Component
Connection	CafeConnectionString (Settings)
ConnectionModifier	Internal
DeleteCommand	(DeleteCommand)
GenerateDBDirectMethods	True
InsertCommand	(InsertCommand)
Modifier	Public
Name	ПродажиTableAdapter
SelectCommand	(SelectCommand)
UpdateCommand	(UpdateCommand)

1) **SelectCommand** — выбрать данные.

Метод **Fill** — заполняет данными набор данных соответствующих таблиц БД.

Он реализует выборку данных из соответствующих таблиц БД.

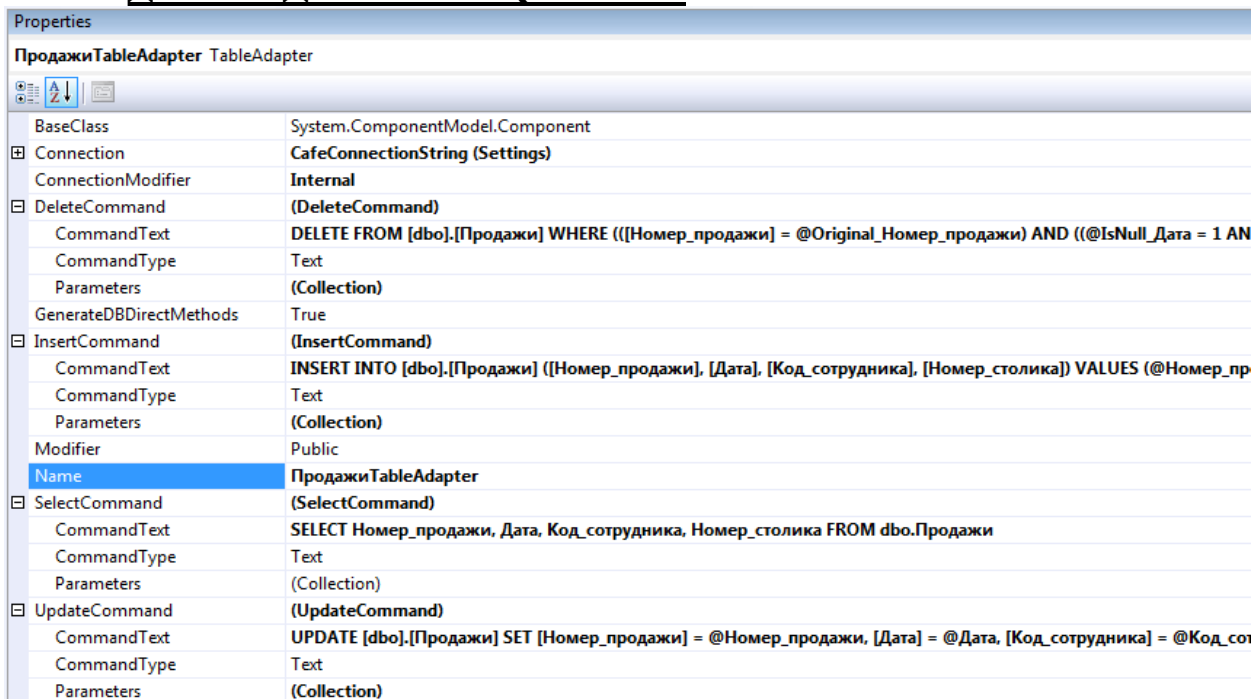
2) **InsertCommand** – добавить данные.

3) **DeleteCommand** — удалить данные.

4) **UpdateCommand** — изменить данные.

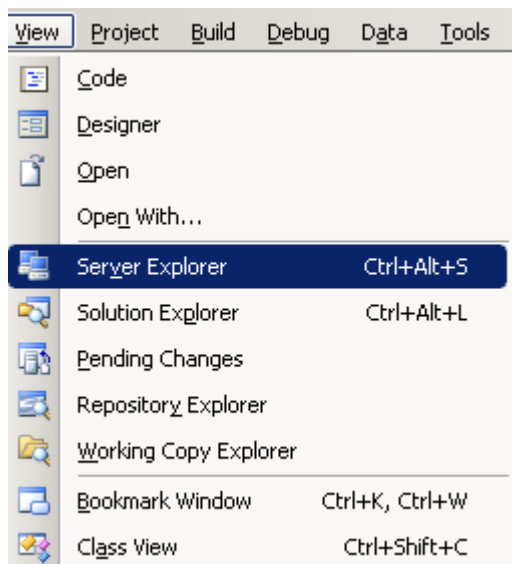
Например, для Продажи:

Для СУБД Microsoft SQL Server:



Properties	
ПродажиTableAdapter TableAdapter	
BaseClass	System.ComponentModel.Component
Connection	CafeConnectionString (Settings)
ConnectionModifier	Internal
DeleteCommand	(DeleteCommand)
CommandText	DELETE FROM [dbo].[Продажи] WHERE (((Номер_продажи) = @Original_Номер_продажи) AND ((@IsNull_Дата = 1 AND
CommandType	Text
Parameters	(Collection)
GenerateDBDirectMethods	True
InsertCommand	(InsertCommand)
CommandText	INSERT INTO [dbo].[Продажи] ([Номер_продажи], [Дата], [Код_сотрудника], [Номер_столика]) VALUES (@Номер_пр
CommandType	Text
Parameters	(Collection)
Modifier	Public
Name	ПродажиTableAdapter
SelectCommand	(SelectCommand)
CommandText	SELECT Номер_продажи, Дата, Код_сотрудника, Номер_столика FROM dbo.Продажи
CommandType	Text
Parameters	(Collection)
UpdateCommand	(UpdateCommand)
CommandText	UPDATE [dbo].[Продажи] SET [Номер_продажи] = @Номер_продажи, [Дата] = @Дата, [Код_сотрудника] = @Код_сот
CommandType	Text
Parameters	(Collection)

Открыв окно **Server Explorer** через меню **View**:



можно просмотреть сформированное соединение к БД.

Примечание:

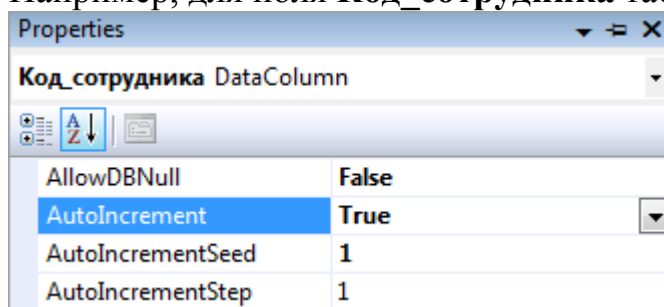
При формировании класса **CafeDataSet** необходимо учесть то, что первичные ключи таблиц Сотрудники, Продажи являются суррогатными и автоматически формируются (ключ со свойством автоинкремент) источником данных.

Это можно обеспечить, задав для ключевых колонок соответствующих таблиц следующие свойства:

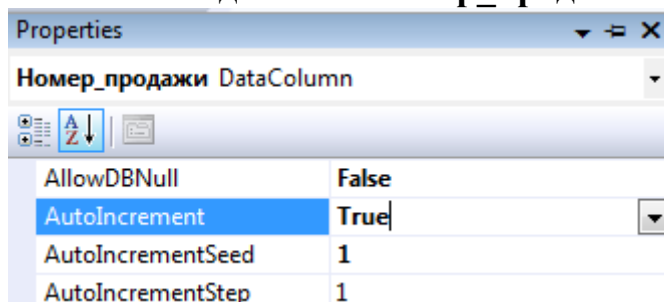
AutoIncrement, AutoIncrementSeed, AutoIncrementStep.

Столбец со свойством **AutoIncrement** равным **true** генерирует последовательность значений, начинающуюся со значения **AutoIncrementSeed** и имеющую шаг **AutoIncrementStep**.

Например, для поля **Код_сотрудника** таблицы **Сотрудники**:



По аналогии для поля **Номер_продажи** в **Продажи**:

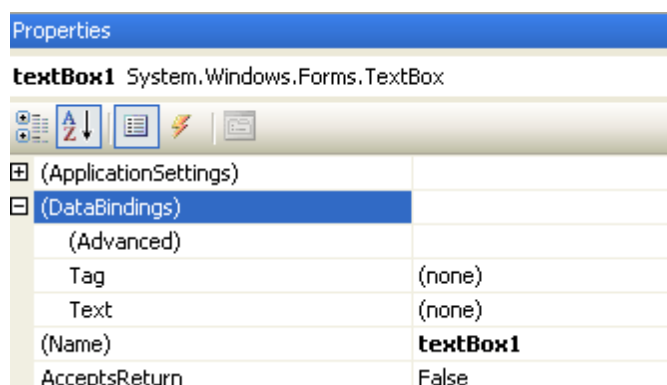


БД и ее объекты подключены в нашем приложении. Перейдем к визуальному оформлению БД в приложении.

4.4 Визуальное оформление БД в приложении

4.4.1 Основные особенности

Многие объекты с палитры **Toolbox** могут работать с данными БД через свойство **DataBindings**, например, для поля/строки ввода (**TextBox**):



По мере выполнения лабораторных работ будем добавлять особенности работы с данными объектами.

Привязывать элементы пользовательского интерфейса можно не только к таблично представленным данным. Практически любая структура данных может выступить в роли их источника – обычные объекты, массивы и т.д.

Для отображения информации из БД на элементах управления Windows Forms необходимо осуществить их заполнение из соответствующих столбцов и строк таблиц DataSet. Это можно сделать путем написания специального кода, который предназначен для заполнения соответствующих элементов управления или использовать механизм привязки данных к пользовательским интерфейсам .NET.

Windows Forms позволяют отображать данные путем привязки их элементов управления к источникам данных. Привязка данных обычно используется для отображения результатов поиска, детализации сводок, генерации отчетов и ввода данных.

Существует два типа привязки данных: простая и сложная. В случае простой привязки данных элемент управления привязывается к отдельному элементу данных. Простая привязка, в основном, используется с такими элементами управления, как поле ввода (**TextBox**), надпись (**Label**) и др. При

сложной привязке данных элемент управления привязывается более чем к одному элементу данных - обычно к одному или нескольким столбцам в нескольких строках результирующего набора строк. К элементам управления, поддерживающим сложную привязку данных, относятся список (**ListBox**), выпадающий список (**ComboBox**), список данных (**DataList**) и табличный элемент (**DataGridView**).

В технологии привязки элементов управления Windows Forms используется абстрактный класс **BindingManagerBase**. Данный класс синхронизирует все элементы управления Windows Forms (т. е. объекты **Binding**), привязанные к одному источнику данных, позволяя отображать информацию об объекте в источнике данных - например, о строке в локальной таблице.

Класс **BindingContext** используется для создания экземпляра объекта **BindingManagerBase**, и возвращается при этом объект **CurrencyManager** или **PropertyManager** в зависимости от типа источника данных.

Компонент BindingSource

Компонент **BindingSource** является универсальным связующим звеном в Windows Forms. Он существенно облегчает задачу привязки данных.

Компонент **BindingSource** можно рассматривать как типизированную коллекцию элементов, при этом тип источника данных может быть задан различными способами.

Компонент **BindingSource** достаточно сложен и предназначен для решения различных задач. Он может применяться как для простой, так и для сложной привязки данных, в качестве одиночного или списочного источника данных, поддерживает фильтрацию, уведомления при изменении списка объектов, транзакционное добавление новых элементов, расширенную поддержку во время разработки и многое другое.

BindingSource представляет собой промежуточный слой между источником данных и соответствующим элементом управления (controlom), к нему привязанным.

Допустим, у нас есть **Label**, **TextBox** и **ComboBox**, привязанные к таблице (**DataTable**). Предположим, что для обновления информации был создан и заполнен данными из БД новый **DataTable**. Встает несложная, в общем то, задача – сменить у всех трех controlов источник данных. Теперь можно изменить свойства **DataSource/DataMember** единственного **BindingSource**, а не у всех controlов.

Покажем, как, например, можно отобразить данные БД в приложении. На примере БД для кафе.

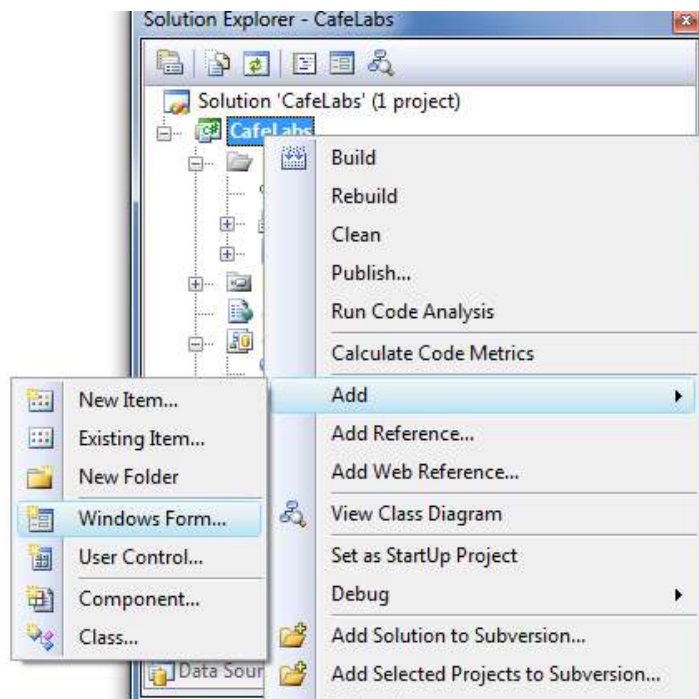
4.4.2 Отображение данных в виде Details

Отобразим данные по блюдам в приложении.

Будем показывать информацию по блюдам в отдельной форме.

Добавим новую форму в проект для отображения данных по блюдам в меню.

Для этого нажмите на проекте правую кнопку мыши и выберите пункт **Add\Windows Forms**:



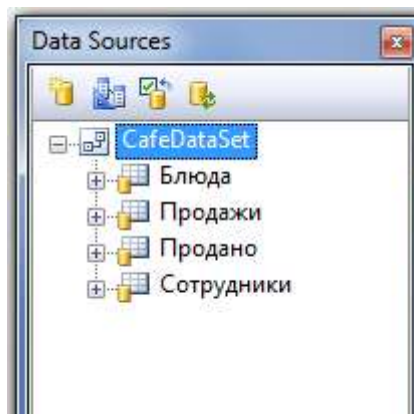
Задайте для новой формы имя **FormDishesList**.

Свойство **Text** для новой формы задайте в **Меню блюд**.

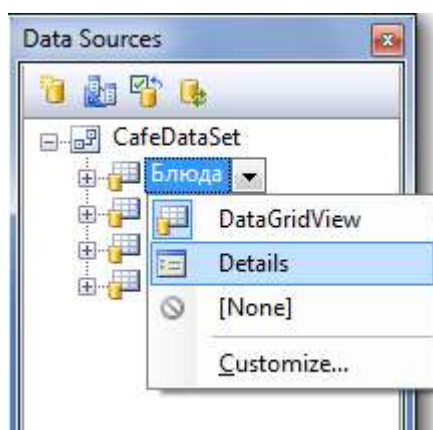


На данной форме будем отображать данные по списку блюд в меню.

Откройте источник данных:

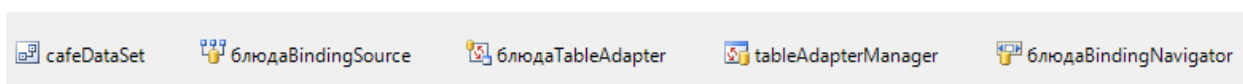


Выберете таблицу **Блюда** и форму представления данных **Details** (каждое поле в отдельном компоненте):



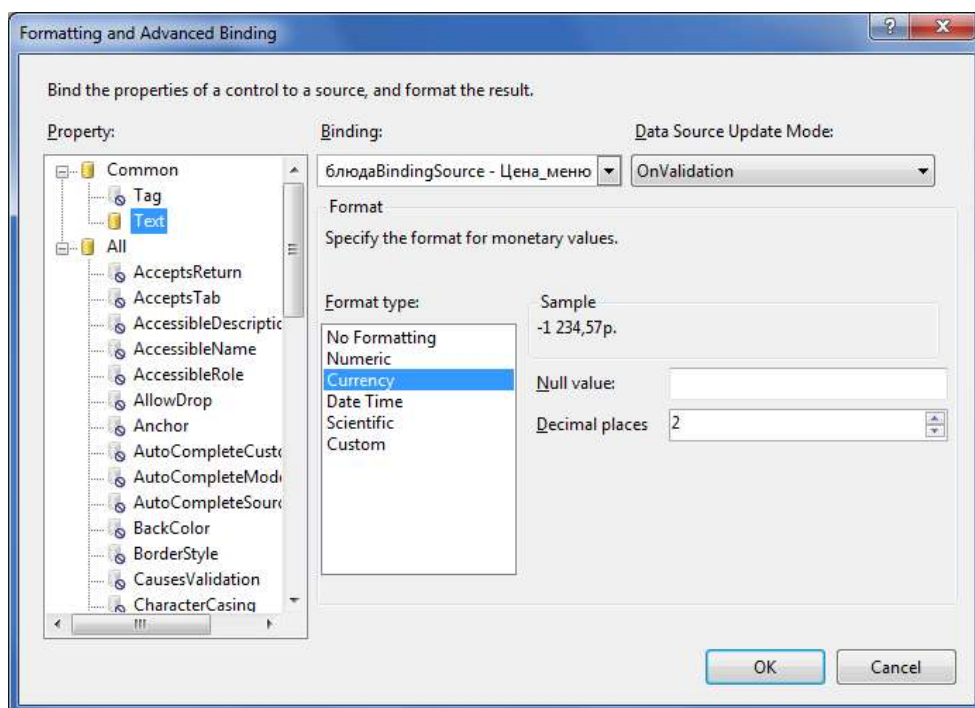
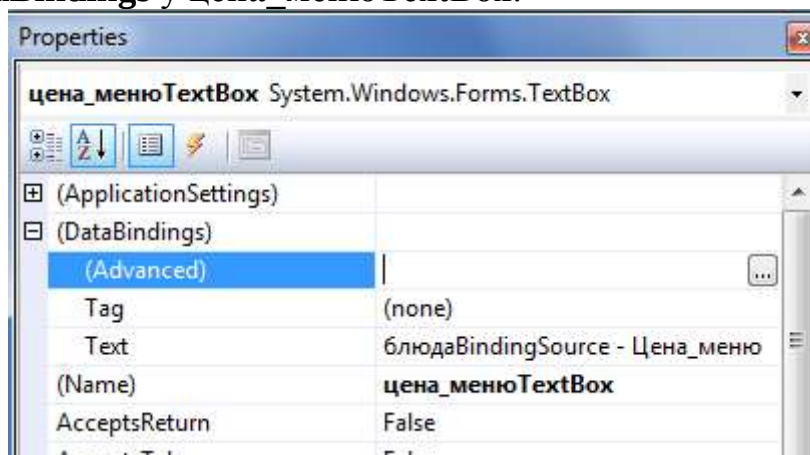
И удерживая левую кнопку мыши, положите на форму для меню блюд (**FormDishesList**). В результате получится форма:

И создадутся объекты:



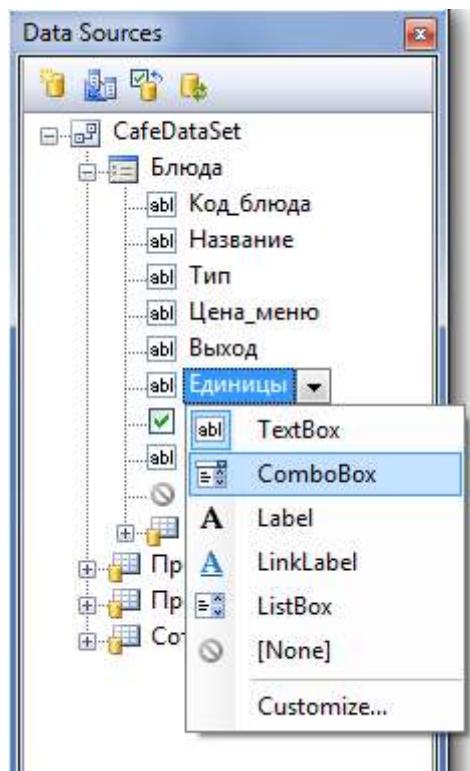
Продолжим настраивать данную форму.

Для данной БД **Цена_меню** должна быть выражена в денежном формате. Для данной настройки укажите в свойстве **Advanced** в **DataBindings** у **цена_менюTextBox**:

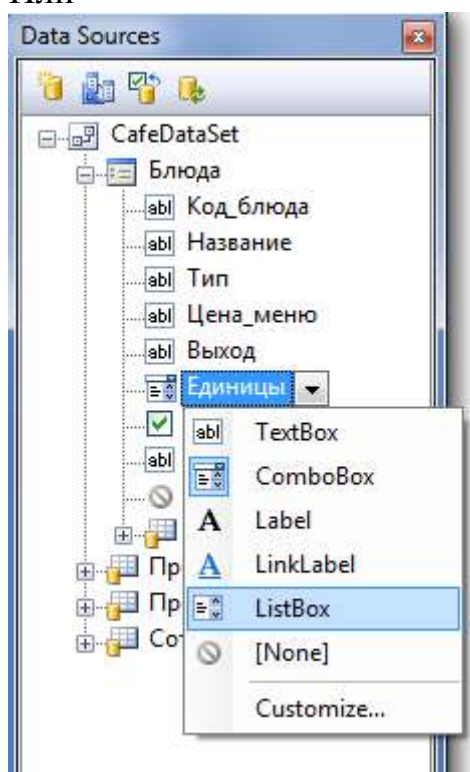


Для поля **Единицы** допустимы значения "гр." или "мл." или "шт." Для удобства пользователей положим или **ComboBox** (выпадающий список) или **ListBox** (список).

Для этого укажите в источнике данных для **Единицы**:

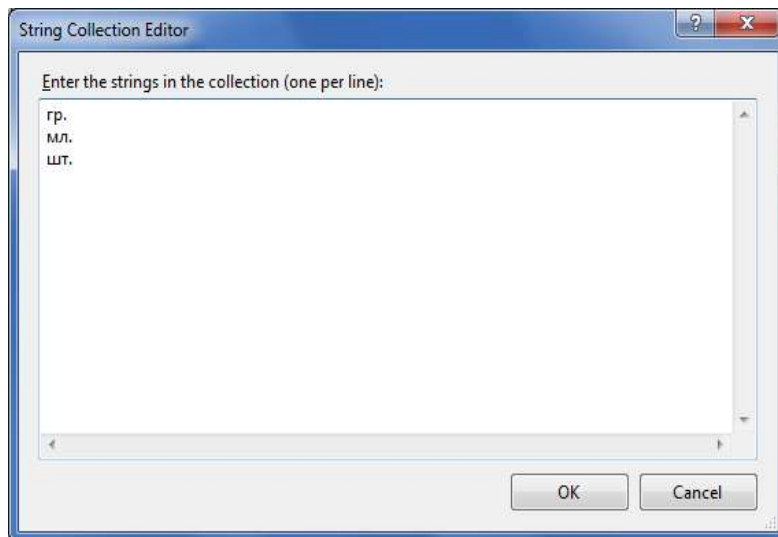


Или



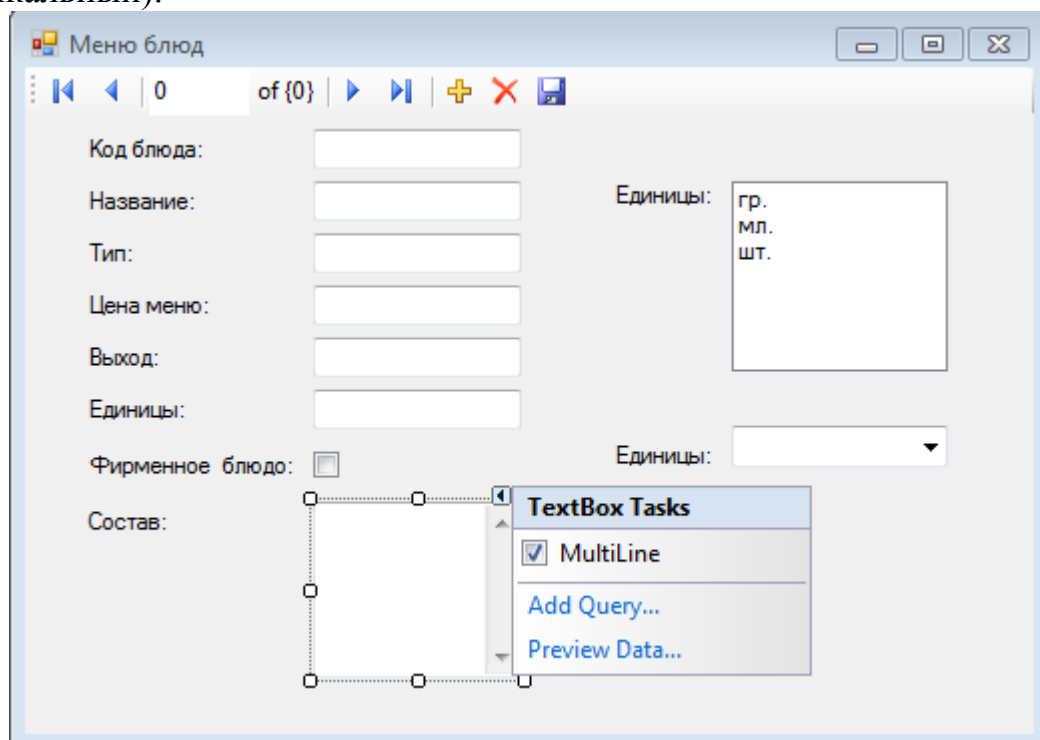
В данном примере для демонстрации разместим оба элемента.

В свойстве **Items** для них укажите:

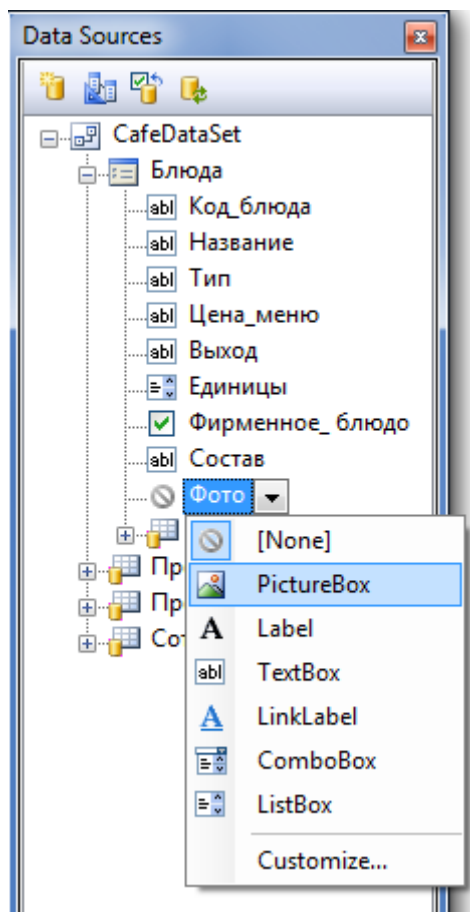


В свойстве **Text** у **фирменное_блюдоCheckBox** укажите пустую строку.

Поле **Состав** может хранить длинный текст. Чтобы отобразить значения в нем несколькими строками укажите свойство **MultiLine** элемента **составTextBox** и растяните его на форме для удобства. Также для удобства установите свойство **ScrollBar** (скролинг) например, в **Vertical** (вертикальный).



Поле **Фото** может хранить фотографии готового блюда. Для их отображения укажите для поля Фото в наборе данных тип элемента **Picture Box**:



И разместите его на форме.

Вид формы имеет вид:

Для **блюдаBindingNavigator** измените на русский язык текст всплывающих подсказок через свойство **Text** для выбранного элемента навигатора. Например, для кнопки с изображением дискеты **блюдаBindingNavigatorSaveItem** укажите свойство **Text** в **Сохранить**.

И далее для остальных кнопок навигатора согласно их смыслу:

	Первая запись, в начало
	Предыдущая запись, назад
	Следующая запись, вперед
	Последняя запись, в конец
	Добавить запись
	Удалить запись
	Сохранить

Для кнопки **bindingNavigatorCountItem** укажите свойство **Text** в **из {0}**

Примечание:

Обратите внимание, что все элементы управления на форме работают через **блюдаBindingSource** (свойство **DataBindings** элементов).

Покажем через главную форму данную форму для блюд.

Перейдите на главную форму и в событии **Click** для пункта меню **Блюда в меню** пропишите:

```
private void блюдаВМенюToolStripMenuItem_Click(object sender, EventArgs e)
{
    FormDishesList fd = new FormDishesList();
    fd.Show();
}
```

Запустите программу и посмотрите, как можно работать с данными через данную форму.

Примечание:

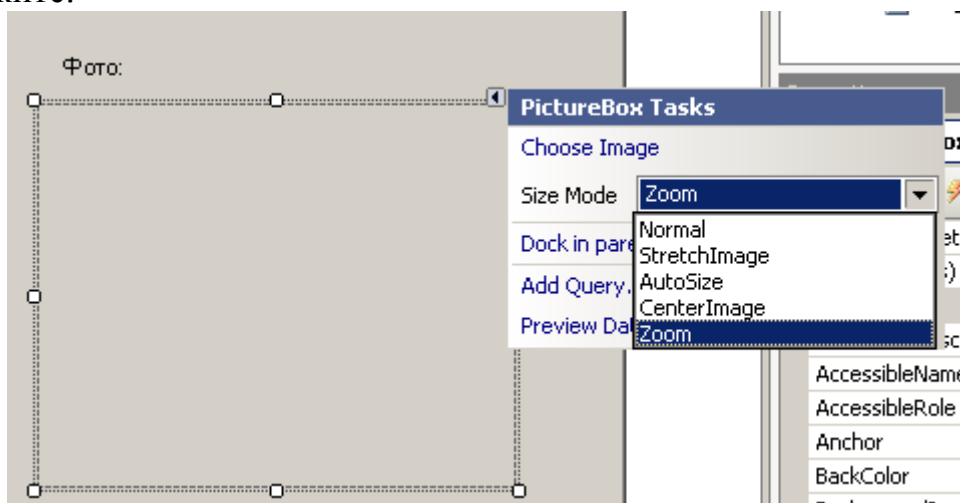
Не забывайте нажимать кнопку Сохранить для сохранения данных.

Закройте исполняемый файл. Перейдите на форму **FormDishesList**.

4.4.3 Работа с графическим полем

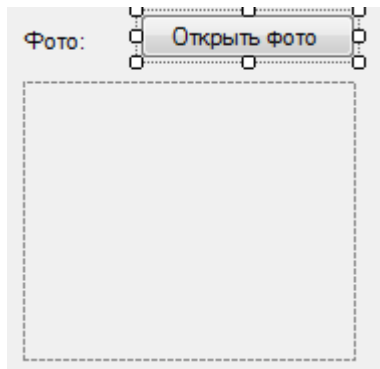
В нашей БД есть поле **Фото** для хранения фотографий блюд.

Чтобы фотография помещалась в размер **фотоPictureBox** для него укажите:



Для удобства загрузки фотографии блюда из файла разместите рядом с **фотоPictureBox** с палитры **Toolbox** компонент **button** для которого укажите свойства:

- **Name** в **buttonOpenPhoto**;
- **Text** в **Открыть фото**.



Для загрузки фото из файла положите на форму с палитры Toolbox диалог **openFileDialog1**. Задайте ему свойство **Name** в **openFileDialogPhoto**.

Запрограммируем нажатие кнопки **buttonOpenPhoto** для загрузки фото в **фотоPictureBox**:

```
string fileImage = "";
private void buttonOpenPhoto_Click(object sender, EventArgs e)
{
    openFileDialogPhoto.Title = "Укажите файл для фото";
    if (openFileDialogPhoto.ShowDialog() == DialogResult.OK)
    {
        fileImage = openFileDialogPhoto.FileName;
        фотоPictureBox.Image = new
Bitmap(openFileDialogPhoto.FileName);
    }
    else fileImage = "";
}
```

Или с конструкцией **try...catch**:

```
string fileImage = "";
private void buttonOpenPhoto_Click(object sender, EventArgs e)
{
    openFileDialogPhoto.Title = "Укажите файл для фото";
    if (openFileDialogPhoto.ShowDialog() == DialogResult.OK)
    {
        fileImage = openFileDialogPhoto.FileName;
        try
        {
            фотоPictureBox.Image = new
Bitmap(openFileDialogPhoto.FileName);
        }
        catch
        {
            MessageBox.Show("Выбран не тот формат файла", "Ошибка",
MessageBoxButtons.OK, MessageBoxIcon.Error);
            return;
        }
    }
    else fileImage = "";
}
```

Конструктор класса **Bitmap** позволяет загрузить файлы следующих форматов: **BMP, GIF, EXIF, JPG, PNG, TIFF**.

Запустите программу, посмотрите, как можно работать с данными через данную форму:

4.4.4 Шаблон проектирования «Одиночка» (англ. Singleton)

Примечание:

Для отображения формы используются два основных метода:

метод Show() – возможен переход между формами;

ShowDialog() – открывает форму в модальном режиме, т.е. невозможно переключиться на другую форму, пока форма, показываемая по ShowDialog() открыта.

Перейдите на главную форму и посмотрите событие **Click** для пункта меню **Блюда в меню**:

```
private void блюдаВМенюToolStripMenuItem_Click(object sender, EventArgs e)
```

```

{
    FormDishesList fd = new FormDishesList();
    fd.Show();
}

```

Запустите программу и посмотрите, как можно работать с данными через данную форму.

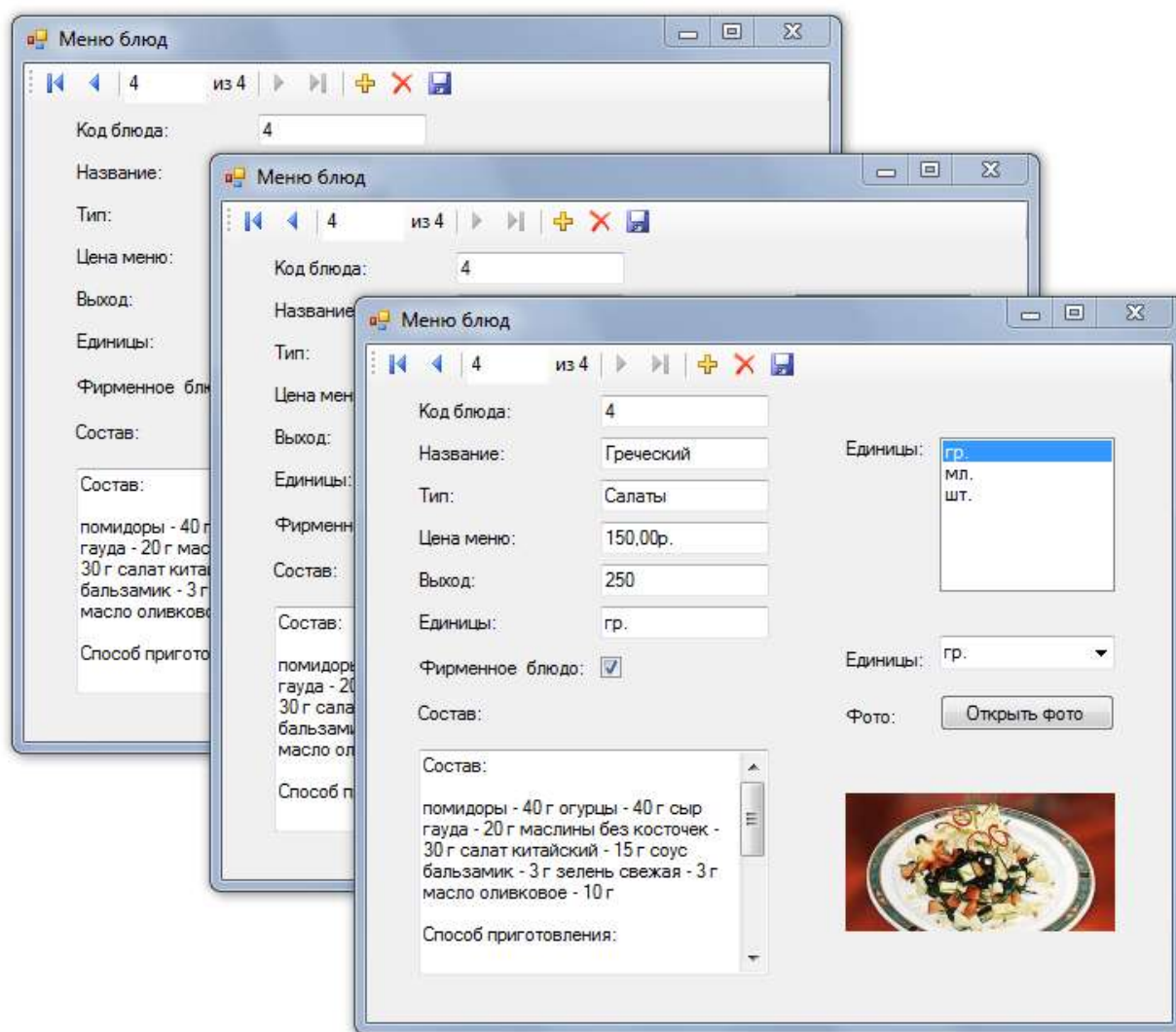
Обратите внимание, что в данный момент нами используется метод **Show()** и возможно переходить от открытой формы для блюд к главной форме и наоборот.

Внимание:

выберете в запущенной программе еще раз данный пункт меню **Блюда в меню**, не закрывая уже открытую форму **Блюда в меню**.

Посмотрите, что создалась еще одна такая же форма.

Таким образом, пользователь может накопить достаточное количество открытых одних и тех же форм.



Закройте программу и вернитесь в Visual Studio.

Пропишите в обработчике для данного обработчика вместо метода **Show()** метод **ShowDialog()**:

```
private void блюдаВМенюToolStripMenuItem_Click(object sender, EventArgs e)
{
    FormDishesList fd = new FormDishesList();
    fd.ShowDialog();
}
```

Запустите программу.

Посмотрите, как можно работать с данными через данную форму.

Внимание:

обратите внимание, что сейчас нельзя перейти в главную форму, пока форма для меню блюд не будет закрыта.

Однако часто пользователям требуется открыть несколько форм одновременно с возможностью переключения между ними. Например, в нашем случае посмотреть по коду блюда его название, тип и другую информацию, не закрывая при этом форму при оформлении продажи.

Получается, что в этом случае лучше использовать метод **Show()**. Но, если пользователь несколько раз выберет соответствующий пункт меню, то несколько одинаковых форм и откроется. Чтобы этого избежать и при выборе этого же пункта переходить к этой же открытой форме можно использовать **шаблон проектирования «Одиночка»** (англ. **Singleton**).

Шаблон проектирования (паттерн, англ. **design pattern**) - это многократно применяемая архитектурная конструкция, предоставляющая решение общей проблемы проектирования в рамках конкретного контекста и описывающая значимость этого решения. Паттерн - это описание или образец для того, как решить задачу, таким образом, чтобы это можно было использовать в различных ситуациях.

Шаблон проектирования «Одиночка» (англ. **Singleton**) - это типичное решение гарантирует, наличие только одного экземпляра класса, и предоставляет ссылку на этот единственный экземпляр (объект).

Реализуем шаблон проектирования «Одиночка» (англ. Singleton) на примере формы FormDishesList.

Для этого в коде формы **FormDishesList** пропишите:

```
private static FormDishesList f;

public static FormDishesList fd
{
    get
    {
        if (f == null || f.IsDisposed) f = new FormDishesList();
        return f;
    }
}
```

Как видно из данного листинга, теперь мы не на пункте меню для блюд каждый раз создаем **fd =new FormDishesList()**, а только в том случае, если она была пустой (**null**) или освобождена в памяти (**IsDisposed**).

Перейдите на главную форму и для соответствующего пункта меню для блюд укажите тогда новый код:

```
private void блюдаВМенюToolStripMenuItem_Click(object sender, EventArgs e)
{
    FormDishesList.fd.Show();
}
```

Запустите программу и посмотрите полученную возможность.

Закройте программу и вернитесь в Visual Studio.

Чтобы при выборе пункта меню для блюд каждый раз возвращать фокус на уже открытую форму (повторно мы ее не создаем) объявим свой метод **ShowForm()** на форме **FormDishesList**:

Для этого в коде формы **FormDishesList** пропишите:

```
public void ShowForm()
{
    Show();
    Activate();
}
```

Тогда на главной форме для пункта меню для блюд пропишите новый код:

```
private void блюдаВМенюToolStripMenuItem_Click(object sender, EventArgs e)
{
    FormDishesList.fd.ShowForm();
}
```

Запустите программу и посмотрите появившуюся возможность.

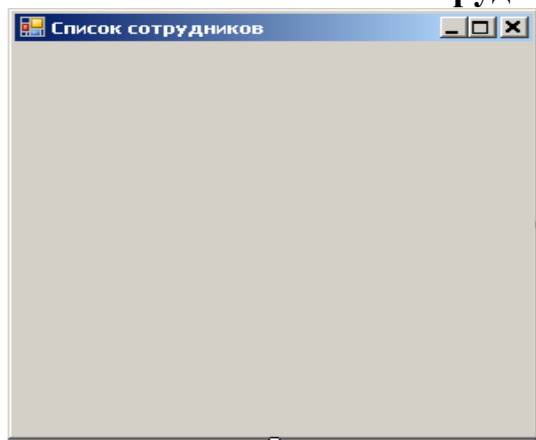
Внимание:

посмотрите, что сейчас возможен переход между формами, в случае повторного выбора пункта меню для блюд будет показана и сфокусирована одна форма для блюд.

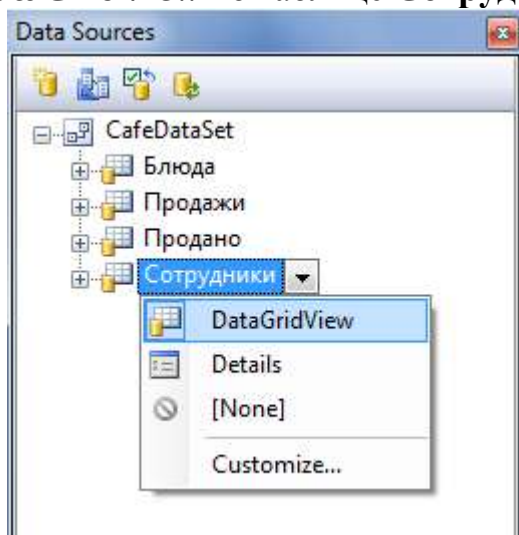
4.4.5 Представление данных в DataGridView

Добавим еще форму в проект для отображения данных по сотрудникам **FormWorkesList**.

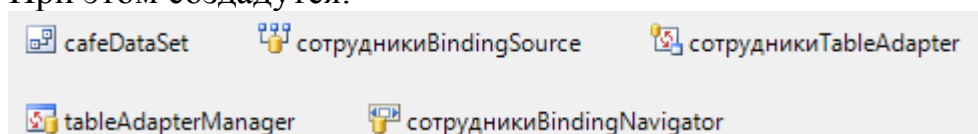
Укажите для нее свойство **Text** в **Список сотрудников**.



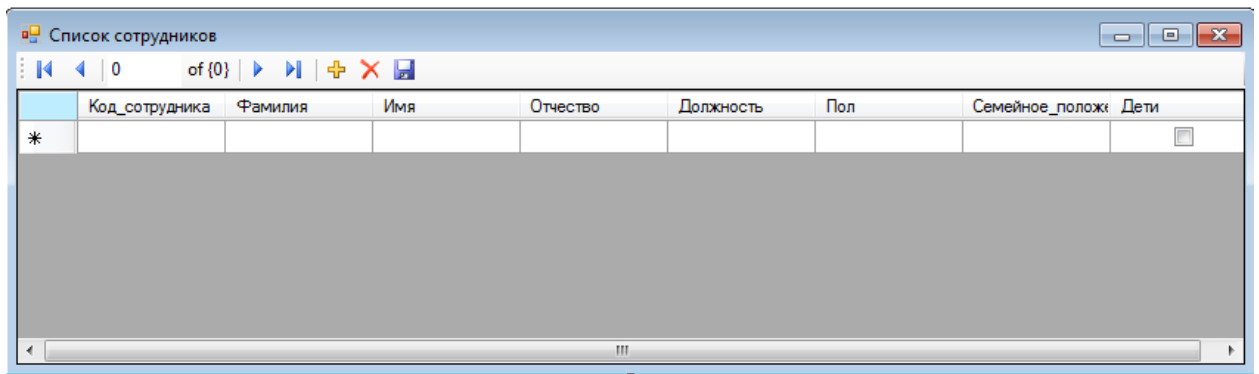
На нее разметите **DataGridView** по таблице **Сотрудники**:



При этом создадутся:



Укажите для **сотрудникиDataGridView** свойство **Dock** в **Fill** (всю область). Вид формы имеет вид:



Для **сотрудникиBindingNavigator** измените на русский язык текст всплывающих подсказок через свойство **Text** для выбранного элемента навигатора. Например, для кнопки с изображением дискеты **сотрудникиBindingNavigatorSaveItem** укажите свойство **Text** в **Сохранить**.

И далее для остальных кнопок навигатора согласно их смыслу.

Для кнопки **bindingNavigatorCountItem** укажите свойство **Text** в **из {0}**

Реализация шаблона проектирования «Одиночка» для формы **FormWorkesList**:

Для формы Сотрудники пропишите в коде **FormWorkesList**:

```
private static FormWorkesList f;

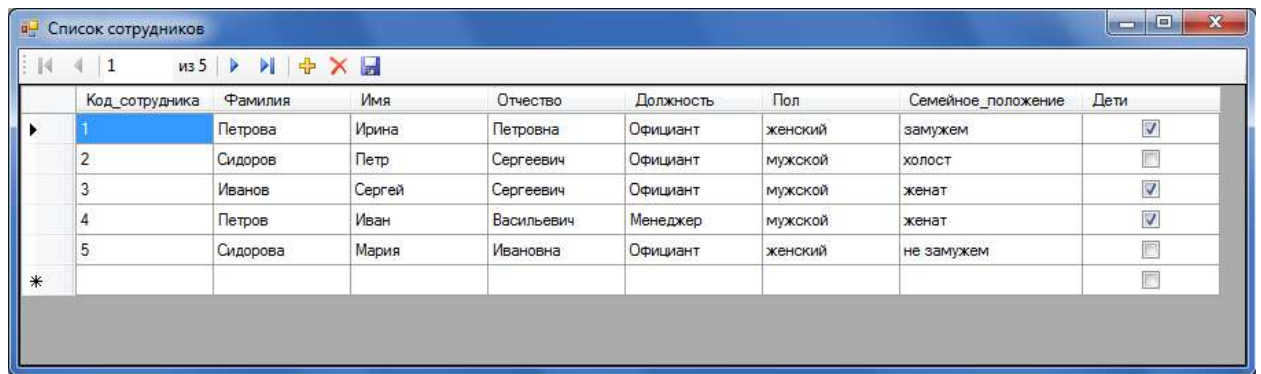
public static FormWorkesList fw
{
    get
    {
        if (f == null || f.IsDisposed) f = new FormWorkesList();
        return f;
    }
}

public void ShowForm()
{
    Show();
    Activate();
}
```

А в коде главной формы для пункта меню Сотрудники пропишите код:

```
private void сотрудникиToolStripMenuItem_Click(object sender, EventArgs e)
{
    FormWorkesList.fw.ShowForm();
}
```

В результате при выборе данного пункта в программе будет показана форма для сотрудников:



Если поместить новый **DataGridView** на форму и не производить никаких настроек, то **DataGridView** будет:

- Автоматически показывать заголовки колонок и заголовки строк. И те, и другие остаются видимыми при любом скроллинге.
- Ставить на одном из заголовков строк маркер (черный треугольничек) текущей строки.
- Выбирать целую строку, если пользователь щелкнет по заголовку строки.
- Выбирать сразу несколько строк, если щелчок по заголовку строки производится с зажатым Ctrl или Shift. При этом текущая строка (помеченная треугольничком) всегда будет единственной.
- Удалять все выбранные строки по нажатию на Delete.
- Отображать ячейку, имеющую фокус ввода, особым образом.
- Если пользователь выполнит двойной щелчок по разделителю колонок, будет произведена автоподборка ширины левой колонки.
- Если в методе Main приложения был вызван метод EnableVisualStyles, будет применяться стиль Windows XP, выбранный в настройках рабочего стола.

Помимо этого **DataGridView** будет поддерживать редактирование содержимого:

- Если пользователь выполнит двойной щелчок по ячейке (или нажмет на ней F2), данная ячейка будет переведена в режим редактирования.
- Если пользователь изменит хотя бы один символ в редактируемой ячейке, на заголовке соответствующей строки появится спецсимвол (пишущий карандаш), и будет отображаться до тех пор, пока фокус ввода не покинет редактируемую ячейку, или пока пользователь не нажмет Esc. Последнее действие восстановит то значение ячейки, которое она содержала до входа в режим редактирования.
- Если пользователь прокрутит control вниз до последней строчки, будет отображена дополнительная, специальная строчка для внесения новой записи. Такая строчка всегда помечена символом звездочки на заголовке.

Когда пользователь любым способом перемещается в эту строку, **DataGridView** добавляет новую запись со значениями по умолчанию. Если

фокус ввода находится в этой строке, и пользователь нажимает Esc, новая запись пропадает, а фокус ввода перемещается на строчку выше.

Если DataGridView привязан через свойство DataSource **к источнику данных**, то по умолчанию выполняется следующее.

- Каждая колонка, получаемая из источника данных, вызовет добавление соответствующей колонки в controlе.
- Названия колонок источника отобразятся в заголовках колонок.
- Если пользователь щелкнет по заголовку колонки, строки будут автоматически отсортированы.

Излишне говорить, что практически все из перечисленного выше может быть разрешено /запрещено /настроено.

Привязка данных в DataGridView может быть проведена несколькими способами:

- Основной – отображение данных из внешних коллекций (например, ListView, DataTable).
- Специальный режим отображения свободных (unbound) данных, то есть данные хранятся в самом controlе.
- Виртуальный (Virtual mode). В нем control посылает событие, при поступлении которого прикладной код возвращает некоторые данные. Так как данные при этом не обязаны где-то храниться, виртуальный режим может оперировать большим количеством строк без каких-либо проблем с производительностью или нехваткой памяти.

В основном DataGridView будет работать в первом режиме, так как в большинстве случаев данные будут поступать из БД, при этом копируясь в промежуточные коллекции, например, DataTable.

Чаще всего данные попадают в DataGridView из подключенного источника данных. При этом встроенный механизм Windows Forms Data Binding автоматически заполняет каждую ячейку значением из соответствующей ячейки источника. Но, как вы уже знаете, DataGridView поддерживает также специальный режим отображения «свободных» (не привязанных, unbound) данных. Кроме того, поддерживается комбинированный режим одновременного отображения связанных и свободных данных.

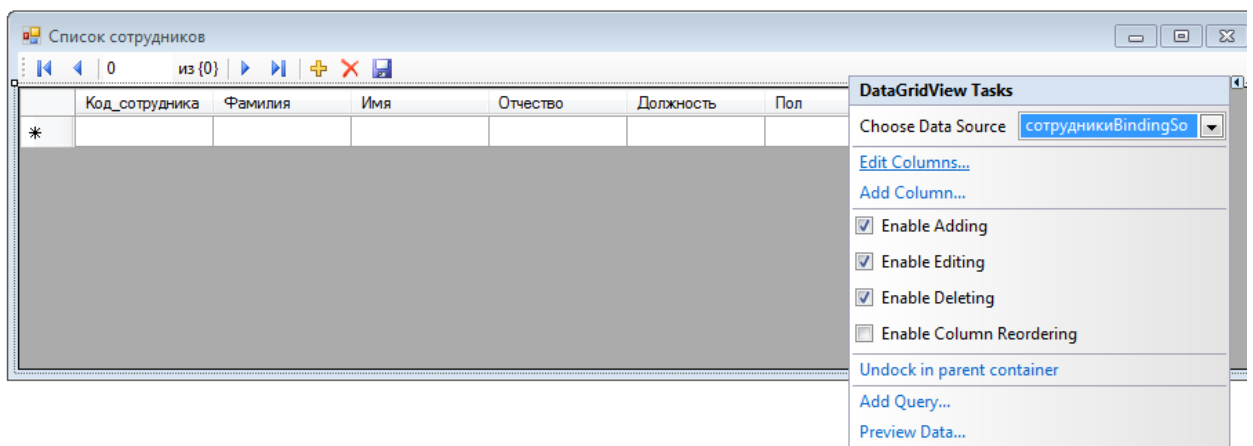
Свободными данными называются данные, заносимые в ячейки вручную. Связанными данными называются данные, заносимые в ячейки автоматически из привязанного источника данных. Будут ли данные считаться связанными или свободными, определяется на уровне колонок. Колонка становится связанной, если в ее свойство DataGridViewColumn.DataPropertyName (типа string) заносится название колонки или свойства объекта из источника данных.

Колонка считается свободной, если упомянутое свойство становится равным пустой строке.

Если свойство `AutoGenerateColumns` выставлено в `true`, `DataGridView` не только генерирует новый список колонок при подключении источника данных, но и привязывает эти колонки к колонкам или свойствам источника данных. Однако это не означает, что нельзя управлять привязкой колонок после их автоматического формирования.

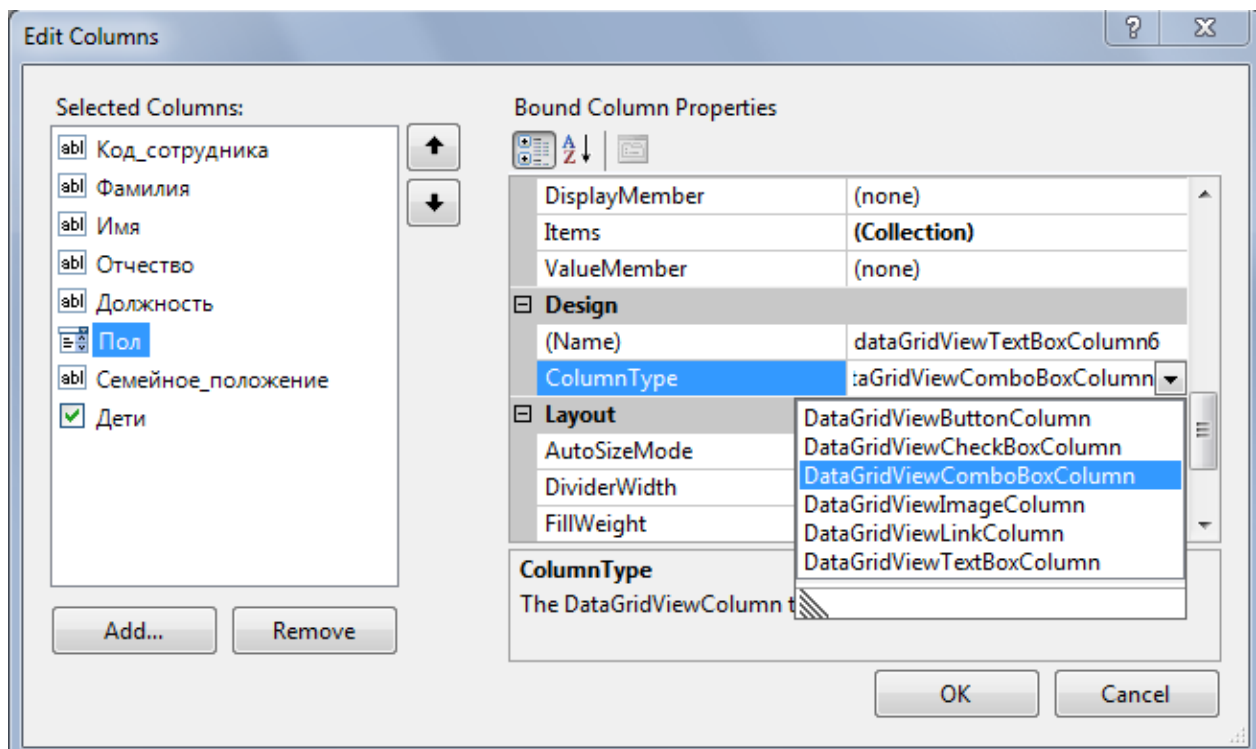
Настроим несколько колонок в `сотрудникиDataGridView` на форме `FormWorkesList`.

Нажмите на `сотрудникиDataGridView` команду **Edit Columns:**

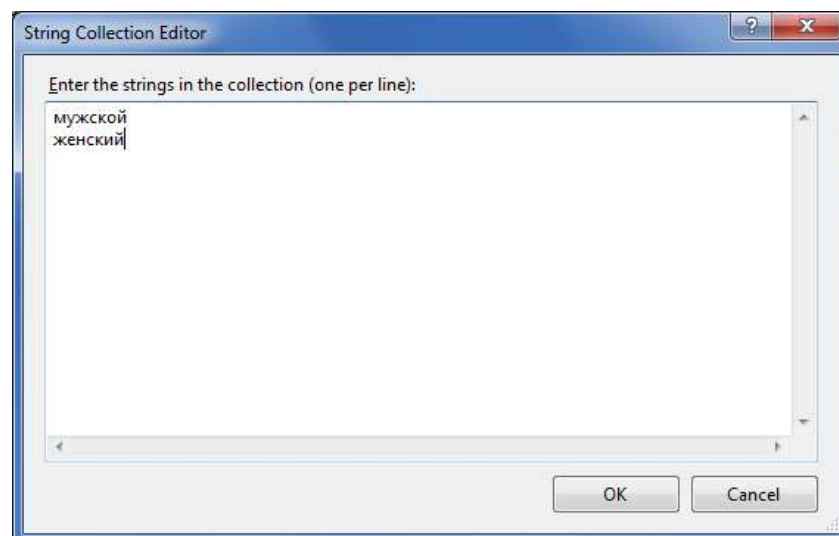


В поле **Пол**, согласно наложенным ограничениям, допустимы значения 'мужской' или 'женский'. Сделаем для удобства работы с этим полем выпадающий список из данных значений (**`DataGridViewComboBoxColumn`**).

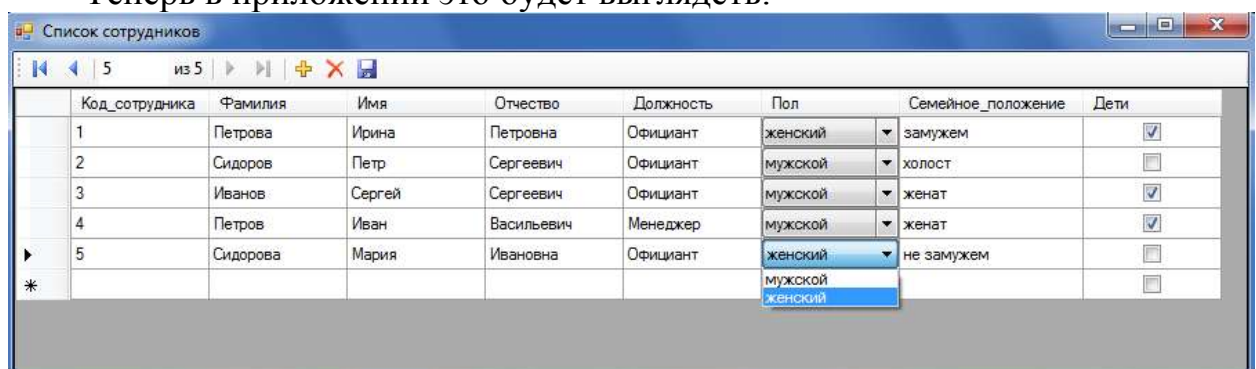
Для поля **Пол** укажите тип колонки:



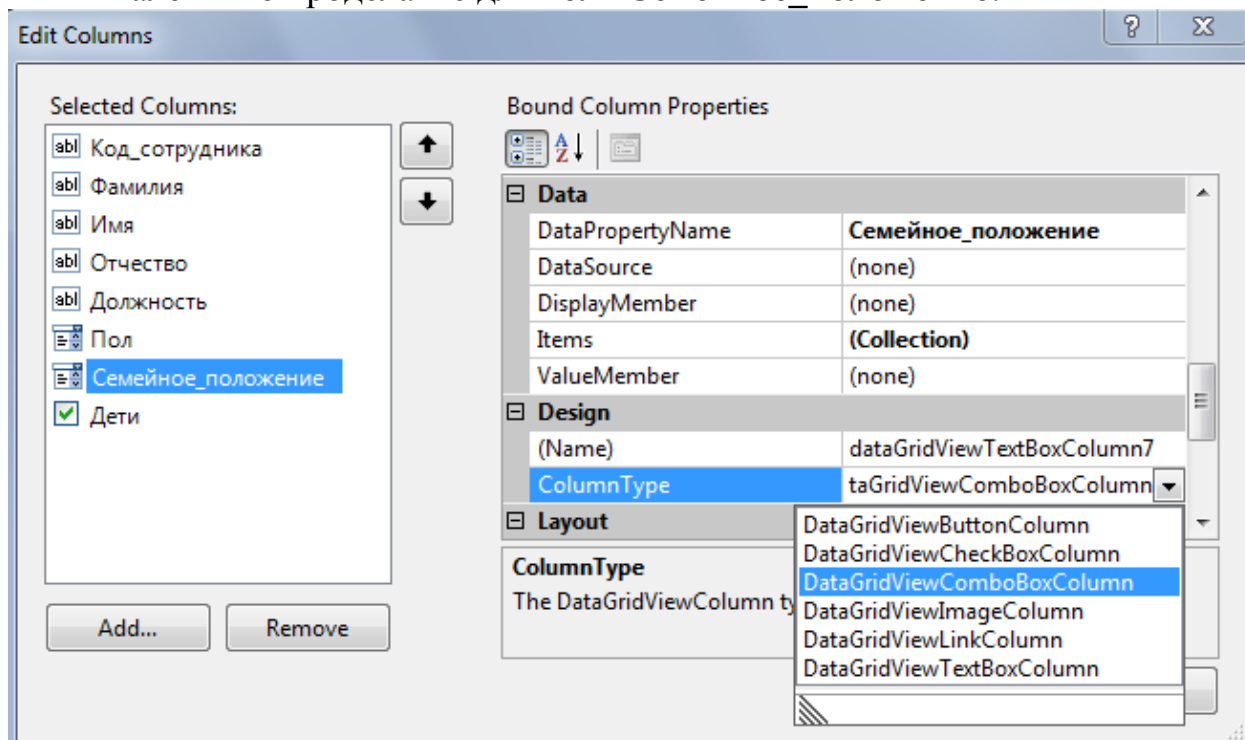
В свойстве **Items** укажите:



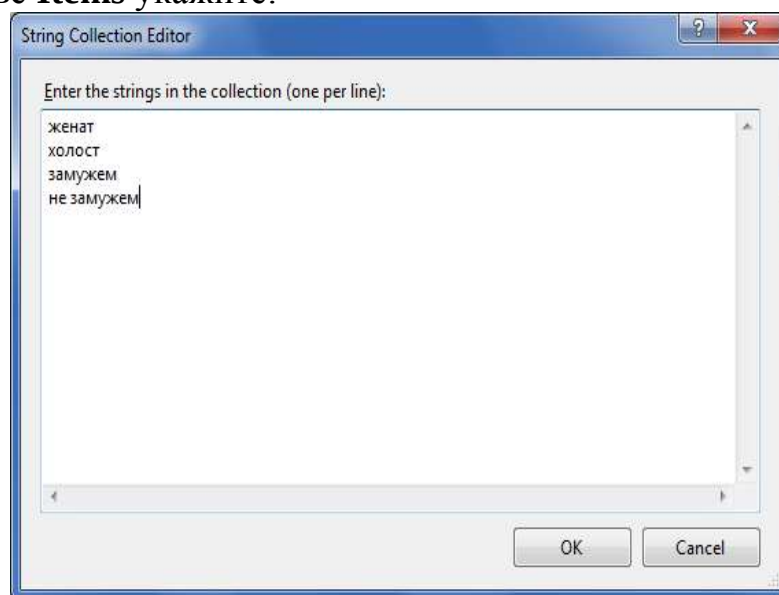
Теперь в приложении это будет выглядеть:



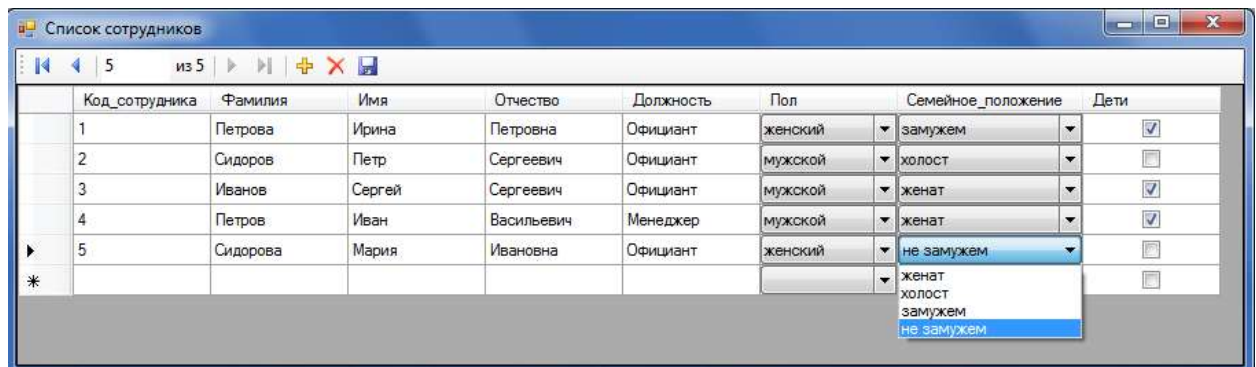
Аналогично проделайте для поля **Семейное_положение**:



В свойстве **Items** укажите:



Вид формы в приложении:



Примечание:

Обратите внимание, что для поля **Дети** из-за выбранного типа данных автоматически задан тип для колонки элемента управления флажок (**DataGridViewCheckBoxColumn**).

Раскрасим сотрудники **DataGridView** по признаку пола сотрудника.

Примечание:

Укажите для дальнейшей работы свойство **Name** для колонки **Пол** в **dataGridViewTextBoxColumnPol**.

Для этого пропишем в событии **CellFormatting** у сотрудников **DataGridView**:

```
private void сотрудникиDataGridView_CellFormatting(object sender,
DataGridViewCellFormattingEventArgs e)
{
    if
    ((сотрудникиDataGridView.Rows[e.RowIndex].Cells["dataGridViewTextBoxColumnPol"]
    .Value == null) ||
    (сотрудникиDataGridView.Rows[e.RowIndex].Cells["dataGridViewTextBoxColumnPol"]
    ].Value.ToString() == ""))
        e.CellStyle.BackColor = Color.LightGreen;
    else
    {
        if
        (сотрудникиDataGridView.Rows[e.RowIndex].Cells["dataGridViewTextBoxColumnPol"]
        ].Value.ToString() == "мужской")
            e.CellStyle.BackColor = Color.SkyBlue;
        else
            e.CellStyle.BackColor = Color.Pink;
    }
}
```

В результате получим раскрашенный сотрудники **DataGridView**:

	Код_сотрудника	Фамилия	Имя	Отчество	Должность	Пол	Семейное_полож	Дети
	1	Петрова	Ирина	Петровна	Официант	женский	замужем	☒
	2	Сидоров	Петр	Сергеевич	Официант	мужской	холост	☐
	3	Иванов	Сергей	Сергеевич	Официант	мужской	женат	☒
	4	Петров	Иван	Васильевич	Менеджер	мужской	женат	☒
	5	Сидорова	Мария	Ивановна	Официант	женский	не замужем	☐
▶	6	Иванов	Егор		Официант			☐
*								☐

Примечание:

В нашей БД к полям Отчество, Пол, Семейное_положение, Дети не были предъявлены требования обязательного заполнения, поэтому такая ситуация, как на рисунке, вполне возможна в данной БД.

Результат можно отсортировать по полу, нажав по заголовку колонки Пол:

	Код_сотрудника	Фамилия	Имя	Отчество	Должность	Пол	Семейное_полож	Дети
	2	Сидоров	Петр	Сергеевич	Официант	мужской	холост	☐
	3	Иванов	Сергей	Сергеевич	Официант	мужской	женат	☒
	4	Петров	Иван	Васильевич	Менеджер	мужской	женат	☒
	1	Петрова	Ирина	Петровна	Официант	женский	замужем	☒
	5	Сидорова	Мария	Ивановна	Официант	женский	не замужем	☐
▶	6	Иванов	Егор		Официант			☐
*								☐

По аналогии можно настраивать данные в приложении и из других объектов БД.

Примечание:

Следует заметить, что в лабораторном практикуме для удобства объяснения материала и понимания предметной области используются идентификаторы с использованием кириллицы. Например, `сотрудникиBindingSource`, `сотрудникиTableAdapter`, `сотрудникиDataGridView`, `сотрудникиBindingNavigator` и др.

Чтобы сократить переключения раскладки клавиатуры при написании исходного текста программы, избежать проблем с возможным переносом программы на другие платформы, не поддерживающие кириллицу в качестве

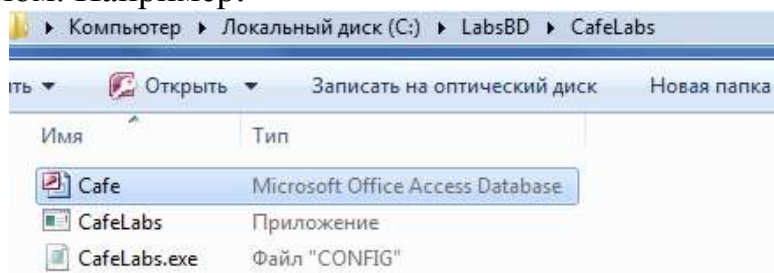
идентификаторов, или при предоставлении исходного кода не русскоязычным разработчикам, предпочтительно использовать латинский/английский алфавит.

4.5 Изменение пути к БД через конфигурационный файл

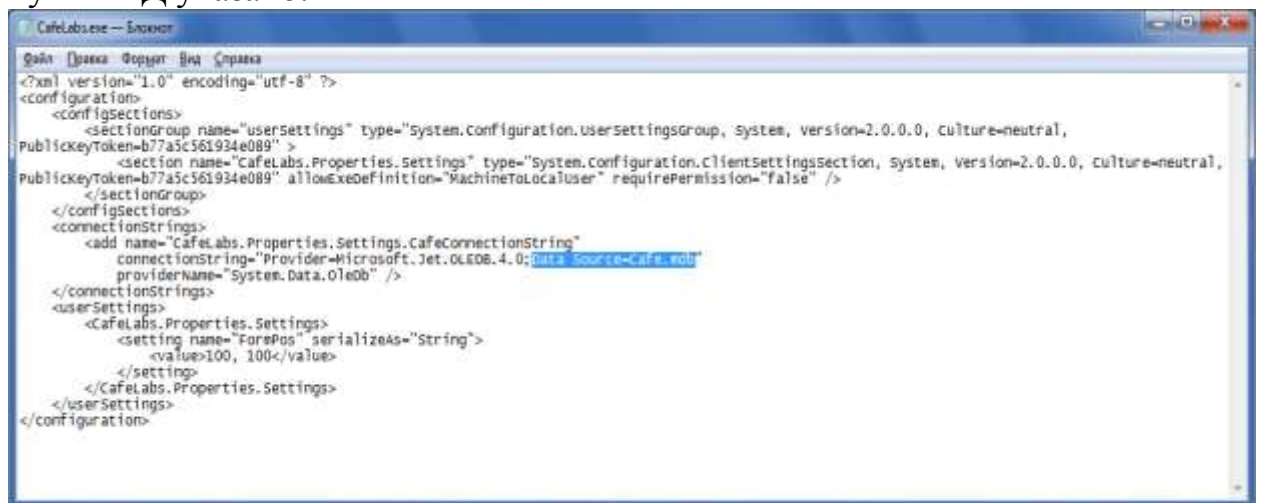
Как было описано ранее (раздел 4.3), мы установили опцию создавать конфигурационный файл, в который занести свойство **ConnectionString** (строка соединения с БД).

Покажем, как менять путь к БД через данный конфигурационный файл.

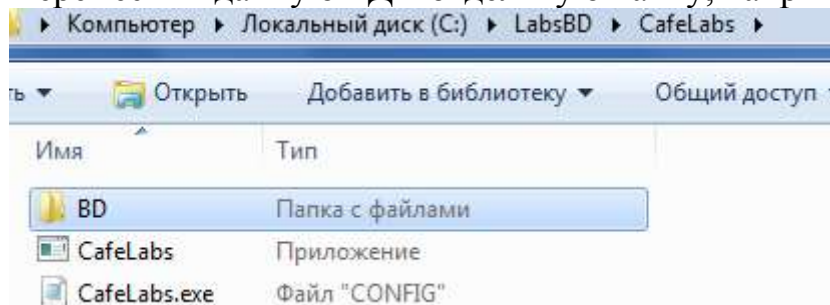
Пусть сейчас, например, файл БД находится в папке рядом с .exe файлом. Например:

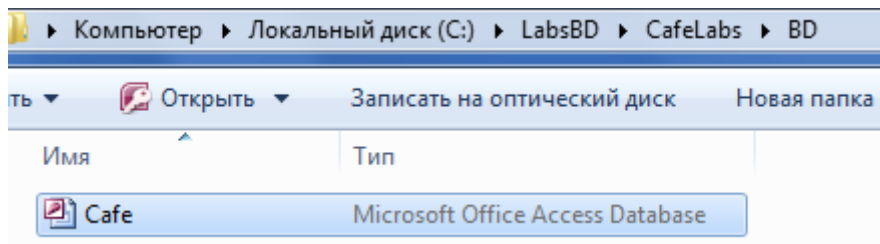


В данном случае в конфигурационном файле CafeLabs.exe.config для пути к БД указано:



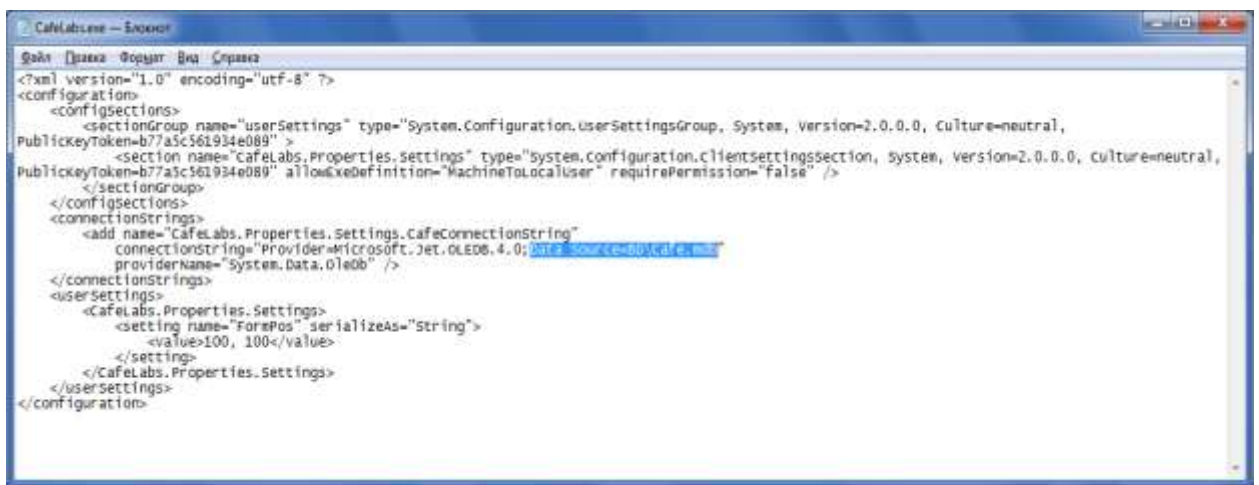
Переместим данную БД в отдельную папку, например, BD:





В данном случае путь к БД изменился. После запуска программы для работы с данными будет выдано сообщение, что не удастся найти файл БД в папке, где .exe файл.

Для устранения данной ошибки откройте конфигурационный файл **CafeLabs.exe.config** и измените в нем путь к БД. В нашем случае – в папке BD рядом с .exe файлом:



В результате, программа по работе с данными вновь будет работоспособной без перекомпиляции приложения.

Примечание:

Если БД установлена на одном компьютере, а приложение на другом, то в этом случае путь к БД указывается через синтаксис:



5 Задание на лабораторную работу №2

5.1 Изучить теоретический и практический материал лабораторной работы №2.

5.2 Для своей БД, созданной в лабораторной работе №1, необходимо выполнить:

5.2.1 создать главную форму проекта с наличием главного меню, контекстного меню, панели инструментов;

5.2.2 настроить, чтобы главная форма сохраняла свое положение при повторном запуске программы;

5.2.3 провести подключение к БД в приложении;

5.2.4 сформировать конфигурационный файл для изменения пути к БД в приложении;

5.2.5 провести визуальное оформление данных БД в приложении. Обязательно:

- наличие нескольких форм в проекте;
- использование различных элементов управления Label, TextBox, ComboBox (и/или ListBox), CheckBox, PictureBox, DataGridView, BindingNavigator и др.

Если в БД нет данных для таких объектов, то изменить структуру БД.

5.2.6 провести модификацию настроек DataGridView (типы колонок обязательно);

5.2.7 настройте для удобства BindingNavigator (всплывающие подсказки обязательно);

5.2.8 предоставить работу с графическим полем (в том числе загрузку фотографии из файла);

5.2.9 реализация шаблона проектирования «Одиночка» (англ. Singleton);

5.2.10 протестируйте работу полученного приложения.

6 Оформление отчёта к лабораторной работе

Оформить отчёт со следующим содержанием:

6.1 Титульный лист.

6.2 Цель работы.

6.3 Постановка задачи.

6.4 Краткая теория и ход выполнения заданий.

6.5 Описание результатов.

Приведите скриншоты полученной программы и листинг программы.

6.6 Заключение (выводы).

Отчет должен быть оформлен согласно требованиям ОС ТУСУР.

7 Контрольные вопросы

- 7.1 Что такое ADO.NET? Ее особенности.
- 7.2 Модель доступа к данным в ADO.NET.
- 7.3 Приведите примеры типов System.Data.
- 7.4 Что такое DataSet? Его особенности.
- 7.5 Особенности DataAdapter.
- 7.6 Как создать главное меню, контекстное меню, панель инструментов?
- 7.7 Как провести подключение БД в приложении Visual Studio?
- 7.8 Для чего нужен компонент BindingSource? Его особенности.
- 7.9 Что характерно для отображения данных в виде Details?
- 7.10 Что характерно для представления данных в данных в DataGridView?
- 7.11 Опишите управляющие кнопки BindingNavigator.
- 7.12 Особенности DataGridView.
- 7.13 Как в DataGridView провести сортировку данных?
- 7.14 Что характерно для шаблона проектирования «Одиночка» (англ. Singleton)?